

Activity No. 1: Number System Converter and Arithmetic Calculator

Name: Wilfred Justin D. Peteros

Course & Section: CPE463-H2

Instructor: Engr. Johnalyn L. Figueras

System Requirements

1. Hardware: A computer or laptop.
2. Software: A modern web browser like Google Chrome or Microsoft Edge.
3. Network: An active internet connection to load the Tailwind CSS framework via CDN and Google Fonts.
4. Tools: A basic text editor such as Notepad for code modification.

Algorithm

1. Start the program.
2. Initialize the user interface with three default input cases, a custom arithmetic expression field, and a dedicated complement subtraction tool.
3. Dynamically assign a sequential alphabetical variable (A, B, C...) to each input case.
4. Prompt the user to enter a number and select its corresponding base (Binary, Octal, Decimal, Hexadecimal) for each active input.
5. Validate each input string against the allowed characters for its selected base, including support for fractional floating-point values.
6. Calculate the standard conversions (Binary, Octal, Decimal, Hexadecimal).
7. Calculate the 1's Complement and 2's Complement for the integer part of each valid input using dynamic bit-padding (8-bit, 16-bit, 32-bit depending on magnitude).
8. Display all six representations (Binary, Octal, Decimal, Hexadecimal, 1's Comp, 2's Comp) in the individual results grid.
9. Convert all valid input strings into a standard decimal floating-point format using a custom fractional parser and map them to their assigned alphabetical variables.
10. Read the custom arithmetic expression input by the user. Parse the expression using the Shunting-yard algorithm to evaluate strict operator precedence and parenthetical grouping logic.
11. Display the formatted equation breakdown and final computed arithmetic result in all base systems and complement formats.
12. Populate the Complement Subtraction Tool with valid variables. When two variables are selected (Minuend and Subtrahend), compute and display the step-by-step binary subtraction process utilizing both the 1's Complement Method (End-Around Carry) and 2's Complement Method.
13. Automatically recalculate and reassign variables (preventing alphabetical gaps) if the user dynamically adds or removes input cases.
14. Handle and display explicit errors for invalid inputs, syntax errors, mismatched parentheses, or mathematical impossibilities (division by zero).
15. End the program.

Pseudocode

```
START PROGRAM
  SET caseCounter = 0
  CALL addCase() THREE TIMES to render initial interface

FUNCTION getComplements(num)
  SET intVal = TRUNCATE(num)
  SET bits = CALCULATE required bits (8, 16, 32) based on intVal
  SET onesComp = INVERT BITS OF (absolute intVal in binary)
  SET twosComp = ADD 1 TO onesComp
  RETURN onesComp, twosComp
END FUNCTION

FUNCTION calculateTotal()
  READ exprString FROM math-expression input
  SET variables = EMPTY MAP

  FOR EACH active case container:
    READ rawValue AND inBase
    IF rawValue IS INVALID THEN ABORT AND DISPLAY ERROR

    SET decValue = CUSTOM PARSE rawValue TO Decimal FLOAT
    STORE decValue IN variables[assignedLetter]

  DISPLAY Base Conversions AND getComplements(decValue)
END FOR

POPULATE Complement Subtraction Selectors WITH keys OF variables

TRY
  SET postfix = SHUNTING_YARD_PARSE(exprString)
  SET finalResult = EVALUATE_POSTFIX(postfix, variables)

  DISPLAY finalResult IN Base 2, 8, 10, 16, 1's Comp, 2's Comp
CATCH ERROR
  DISPLAY explicit error message
END TRY
END FUNCTION

FUNCTION calcComplementSub()
  READ minuendVar AND subtrahendVar
  SET M = TRUNCATE(variables[minuendVar])
  SET S = ABS(TRUNCATE(variables[subtrahendVar]))

  SET paddedM = BINARY(M) padded to common bit length
  SET paddedS = BINARY(S) padded to common bit length

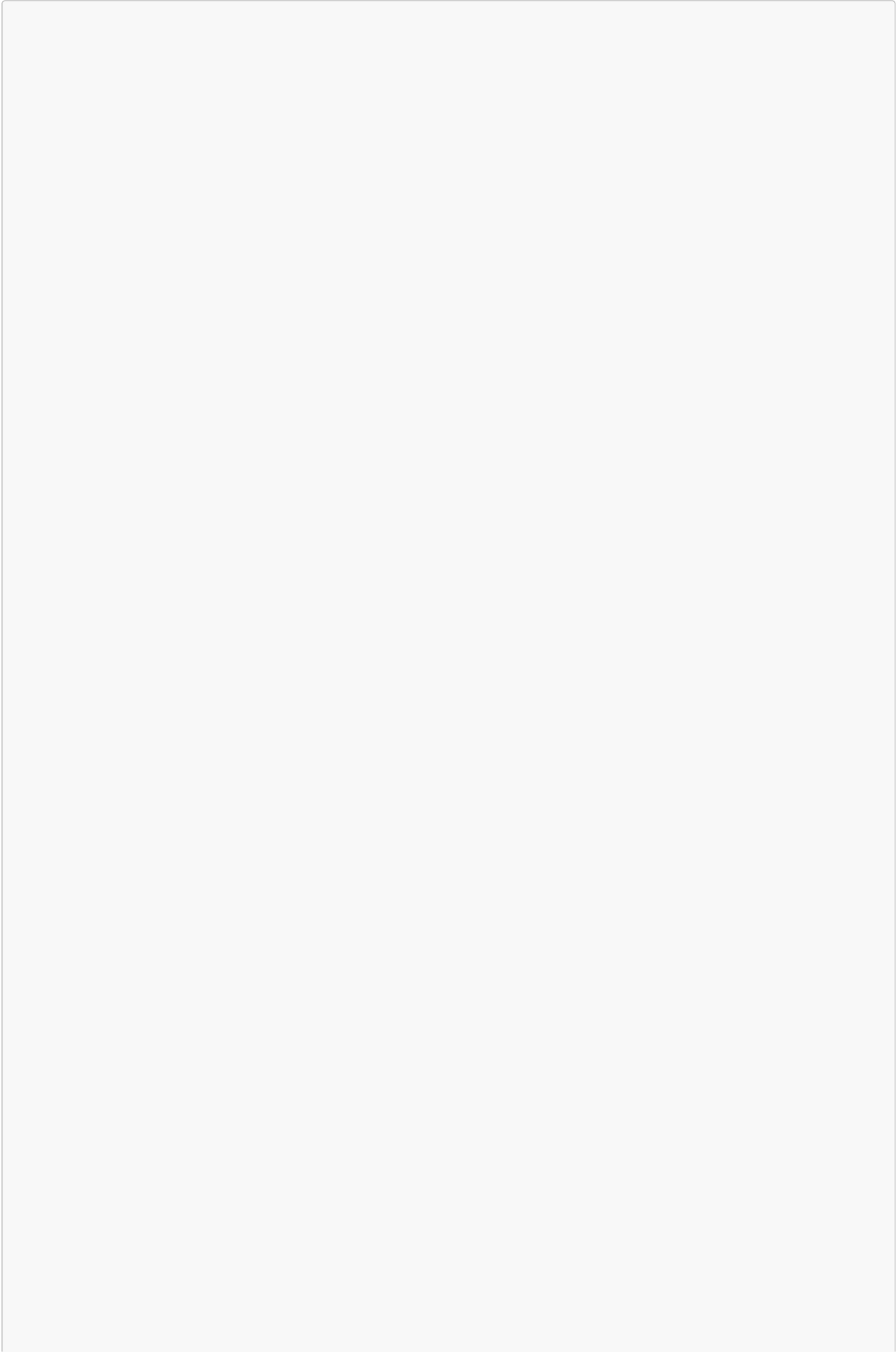
  // 1's Complement Method
  SET onesS = INVERT(paddedS)
  SET sum1 = paddedM + onesS
  IF carry THEN sum1 = sum1 + 1 (End Around Carry)

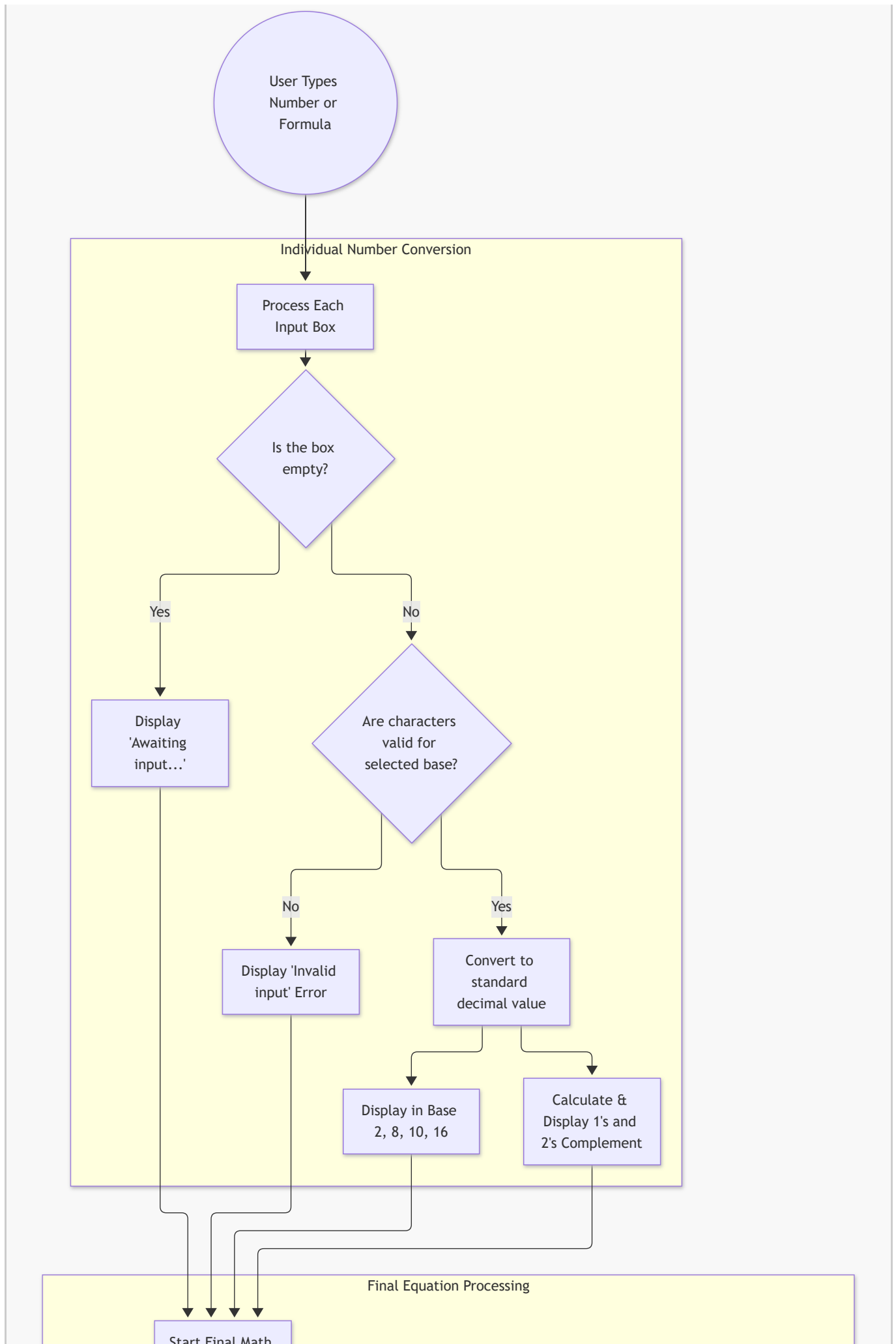
  // 2's Complement Method
```

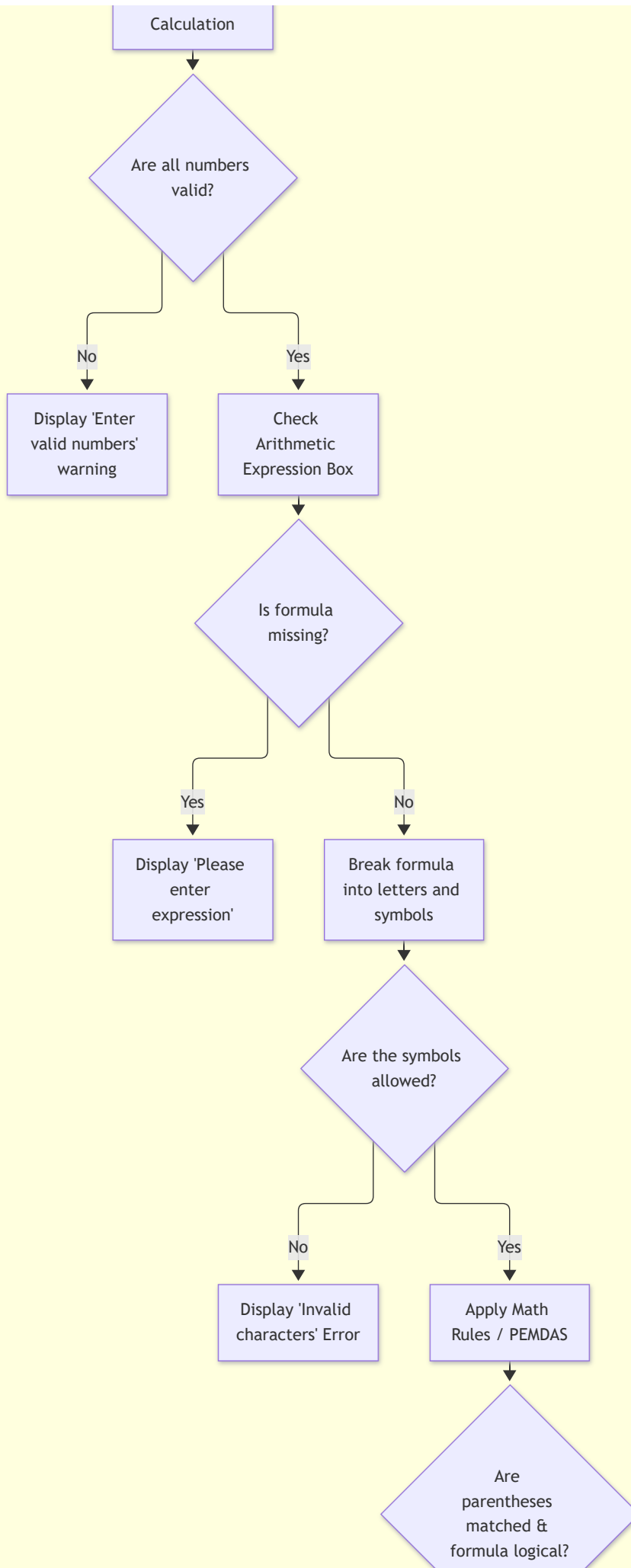
```
SET twosS = INVERT(paddedS) + 1
SET sum2 = paddedM + twosS (Discard Carry)

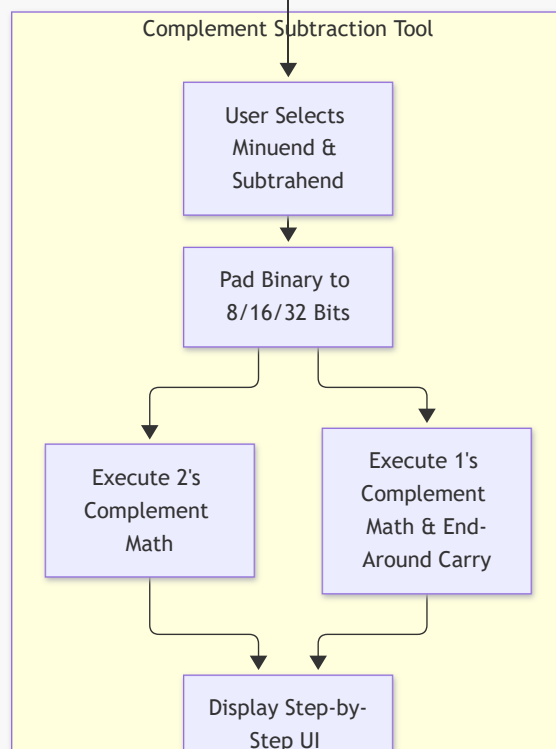
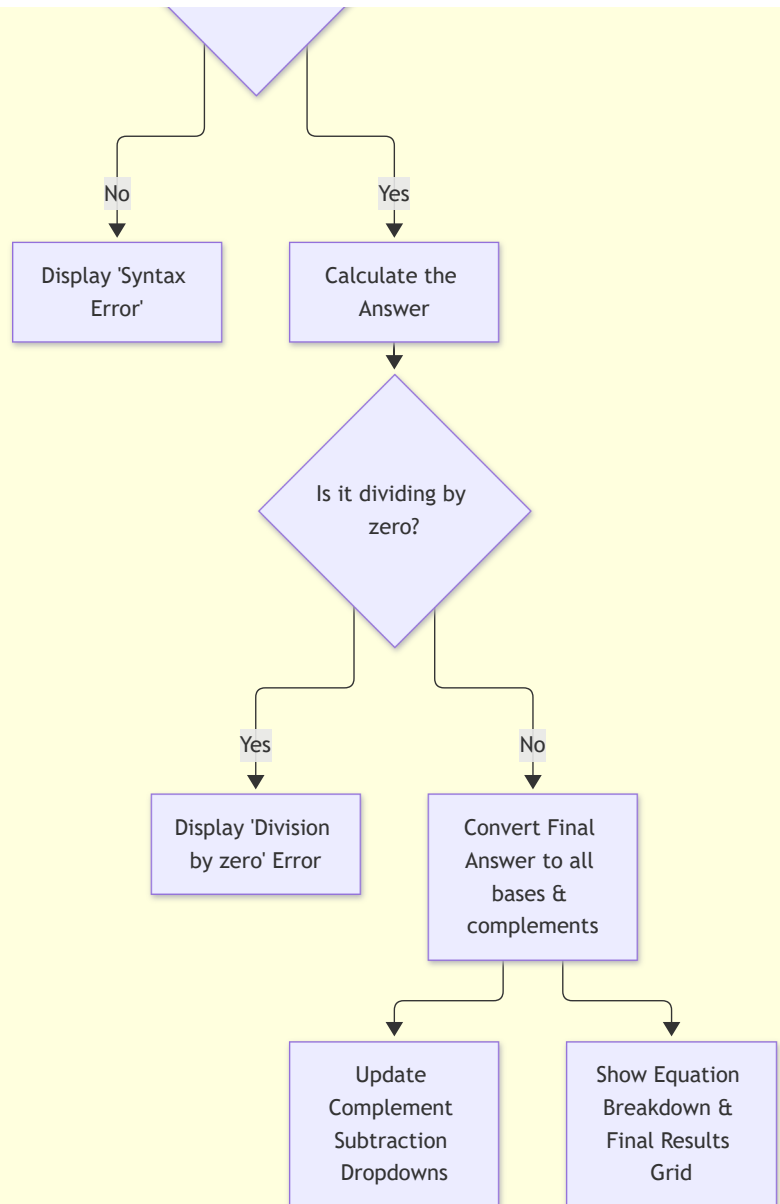
DISPLAY step-by-step calculations FOR both methods
END FUNCTION
END PROGRAM
```

Flowchart









Breakdown

Program Implementation

```

<!DOCTYPE html>
<html lang="en" class="dark">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Number System Converter & Calculator</title>

  <link rel="preconnect" href="[https://fonts.googleapis.com]
(https://fonts.googleapis.com)">
  <link rel="preconnect" href="[https://fonts.gstatic.com]
(https://fonts.gstatic.com)" crossorigin>
  <link href="[https://fonts.googleapis.com/css2?
family=Plus+Jakarta+Sans:wght@400;600;800&family=Space+Grotesk:wght@500;700&displa
y=swap](https://fonts.googleapis.com/css2?
family=Plus+Jakarta+Sans:wght@400;600;800&family=Space+Grotesk:wght@500;700&displa
y=swap)" rel="stylesheet">

  <script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)">
</script>
  <script>
    tailwind.config = {
      darkMode: 'class',
      theme: {
        extend: {
          fontFamily: {
            sans: ['"Plus Jakarta Sans"', 'sans-serif'],
            mono: ['"Space Grotesk"', 'monospace'],
          }
        }
      }
    }
  </script>
</head>
<body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-
slate-800 dark:text-indigo-100 transition-colors duration-300">

  <div class="max-w-6xl mx-auto">
    <div class="flex justify-end mb-4">
      <button onclick="toggleTheme()" class="p-3 rounded-full shadow-lg bg-indigo-
100 dark:bg-slate-800 text-indigo-700 dark:text-purple-400 hover:bg-indigo-200
dark:hover:bg-slate-700 border border-indigo-200 dark:border-indigo-800
transition-colors focus:outline-none focus:ring-2 focus:ring-indigo-500"
title="Toggle Light/Dark Mode">
        <svg class="w-6 h-6 hidden dark:block" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="[http://www.w3.org/2000/svg]
(http://www.w3.org/2000/svg)">

```

```

        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M12 3v1m0 16v1m9-9h-1M4 12H3m15.364 6.364l-.707-.707M6.343
6.343l-.707-.707m12.728 0l-.707.707M6.343 17.657l-.707.707M16 12a4 4 0 11-8 0 4 4
0 018 0z"></path>
    </svg>
    <svg class="w-6 h-6 block dark:hidden" fill="none" stroke="currentColor"
viewBox="0 0 24 24" xmlns="[http://www.w3.org/2000/svg]
(http://www.w3.org/2000/svg)">
        <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2"
d="M20.354 15.354A9 9 0 018.646 3.646 9.003 9.003 0 0012 21a9.003 9.003 0 008.354-
5.646z"></path>
    </svg>
</button>
</div>

<div class="bg-white dark:bg-slate-900 p-6 md:p-8 rounded-2xl shadow-xl border
border-slate-200 dark:border-indigo-900 transition-colors duration-300">
    <h1 class="text-3xl md:text-4xl font-extrabold mb-8 text-center text-indigo-
700 dark:text-purple-400 font-mono tracking-tight">System Converter &
Calculator</h1>

    <div class="mb-8 p-6 bg-indigo-50 dark:bg-indigo-950/30 border border-
indigo-200 dark:border-indigo-800 rounded-xl">
        <label class="block text-lg font-bold text-indigo-800 dark:text-purple-300
mb-3 text-center">Arithmetic Expression</label>
        <p class="text-sm text-center text-indigo-600 dark:text-indigo-400 mb-
4">Use variables (A, B, C...) and operators (+, -, *, /, parentheses). Example:
<strong>(A + B - C) * D</strong></p>
        <input type="text" id="math-expression" class="w-full md:w-3/4 mx-auto
block bg-white dark:bg-slate-900 border border-indigo-300 dark:border-indigo-700
p-4 rounded-lg text-2xl text-slate-900 dark:text-white font-bold text-center
focus:outline-none focus:ring-4 focus:ring-indigo-500 dark:focus:ring-purple-500
transition-colors shadow-sm uppercase tracking-widest" placeholder="A + B - C"
value="A + B - C" oninput="calculateTotal()">
    </div>

    <div id="cases-container" class="grid grid-cols-1 gap-6">
        <!-- Dynamic cases injected here -->
    </div>

    <div class="mt-6 text-center">
        <button onclick="addCase()" class="bg-indigo-600 hover:bg-indigo-700
dark:bg-slate-800 dark:hover:bg-slate-700 border border-transparent dark:border-
indigo-700 text-white dark:text-purple-300 font-bold py-3 px-8 rounded-xl
transition-colors shadow-lg focus:outline-none focus:ring-2 focus:ring-indigo-
500">
            + Add Input Number
        </button>
    </div>

    <div class="mt-12 p-8 bg-slate-900 dark:bg-black border-2 border-indigo-500
dark:border-purple-600 rounded-2xl shadow-2xl relative overflow-hidden">
        <div class="absolute top-0 left-0 w-full h-1 bg-gradient-to-r from-indigo-
500 via-purple-500 to-pink-500"></div>

```

```

    <h2 class="text-2xl font-bold text-white mb-6 text-center">Final
Arithmetic Result</h2>
    <div id="final-total-container" class="text-center">
        <p class="text-slate-400 text-lg">Enter valid numbers and a proper
expression above to compute the result.</p>
    </div>
</div>

<div class="mt-8 p-8 bg-slate-800 dark:bg-slate-950 border-2 border-emerald-
500 dark:border-emerald-600 rounded-2xl shadow-xl relative overflow-hidden">
    <div class="absolute top-0 left-0 w-full h-1 bg-gradient-to-r from-
emerald-400 to-teal-500"></div>
    <h2 class="text-2xl font-bold text-white mb-2 text-center">Complement
Subtraction Method</h2>
    <p class="text-sm text-center text-emerald-400 mb-6">Select two variables
to demonstrate step-by-step subtraction (Minuend - Subtrahend) using complements.
Uses absolute integer parts.</p>

    <div class="flex justify-center items-center gap-4 mb-6">
        <select id="comp-minuend" class="bg-slate-700 text-white p-3 rounded-lg
border border-slate-600 focus:ring-2 focus:ring-emerald-500 font-bold text-lg"
onchange="calcComplementSub()"></select>
        <span class="text-white text-3xl font-bold">-</span>
        <select id="comp-subtrahend" class="bg-slate-700 text-white p-3 rounded-
lg border border-slate-600 focus:ring-2 focus:ring-emerald-500 font-bold text-lg"
onchange="calcComplementSub()"></select>
    </div>

    <div id="comp-sub-output" class="text-slate-300 text-center font-mono">
        Awaiting variable selection...
    </div>
</div>

</div>
</div>

<script>
    let caseCounter = 0;
    window.globalVars = {};

    function toggleTheme() {
        document.documentElement.classList.toggle('dark');
    }

    function parseToDecimal(str, base) {
        const isNegative = str.startsWith('-');
        const cleanStr = str.replace('-', '');
        const parts = cleanStr.split('.');

        let intPart = parseInt(parts[0] || '0', base);
        let fracPart = 0;

        if (parts.length > 1) {
            let fracStr = parts[1];

```

```

    let divisor = base;
    for (let i = 0; i < fracStr.length; i++) {
        fracPart += parseInt(fracStr[i], base) / divisor;
        divisor *= base;
    }
}

let total = intPart + fracPart;
return isNegative ? -total : total;
}

function formatBase(num, base) {
    if (isNaN(num)) return "NaN";
    const isNeg = num < 0;
    const absVal = Math.abs(num);
    let str = absVal.toString(base).toUpperCase();
    return (isNeg ? '-' : '') + str;
}

function getComplements(num) {
    if (isNaN(num)) return { ones: "NaN", twos: "NaN" };
    let intVal = Math.trunc(Math.abs(num));
    let binStr = intVal.toString(2);

    let bits = 8;
    while (binStr.length > bits - 1) bits += 8;
    binStr = binStr.padStart(bits, '0');

    let onesComp = binStr.split('').map(b => b === '0' ? '1' : '0').join('');

    let twosComp = '';
    let carry = 1;
    for (let i = bits - 1; i >= 0; i--) {
        let sum = parseInt(onesComp[i]) + carry;
        twosComp = (sum % 2) + twosComp;
        carry = Math.floor(sum / 2);
    }

    return { ones: onesComp, twos: twosComp, bits: bits };
}

function addBinaryStr(a, b) {
    let res = '', carry = 0;
    for (let i = a.length - 1; i >= 0; i--) {
        let sum = parseInt(a[i]) + parseInt(b[i]) + carry;
        res = (sum % 2) + res;
        carry = Math.floor(sum / 2);
    }
    return { sum: res, carry: carry };
}

function createCaseHTML(id) {
    return `
        <div id="case-${id}" data-var="" class="p-6 border border-slate-200

```

```

dark:border-indigo-800 rounded-xl bg-slate-50 dark:bg-slate-800 relative shadow-sm
dark:shadow-inner transition-colors duration-300">
    <div class="flex justify-between items-center mb-4">
        <label class="case-label font-bold text-xl text-indigo-800 dark:text-
purple-300 bg-indigo-100 dark:bg-indigo-900/50 px-4 py-1 rounded-full border
border-indigo-200 dark:border-indigo-700">Input</label>
        <button onclick="removeCase(${id})" class="remove-btn p-2 text-red-500
dark:text-red-400 hover:text-red-700 dark: hover:text-red-300 hover:bg-red-100
dark: hover:bg-red-900/30 rounded-lg hidden transition-colors focus:outline-none"
title="Remove Input">
            <svg class="w-5 h-5" fill="none" stroke="currentColor" viewBox="0 0
24 24" xmlns="http://www.w3.org/2000/svg" (http://www.w3.org/2000/svg)">
                <path stroke-linecap="round" stroke-linejoin="round" stroke-
width="2" d="M6 18L18 6M6 6l12 12"></path>
            </svg>
        </button>
    </div>

    <div class="grid grid-cols-1 md:grid-cols-2 gap-4 mb-4">
        <div>
            <label class="block text-sm font-semibold text-slate-600 dark:text-
indigo-300 mb-1">Value</label>
            <input type="text" id="val-${id}" class="w-full bg-white dark:bg-
slate-900 border border-slate-300 dark:border-indigo-700 p-3 rounded-lg text-lg
font-mono text-slate-900 dark:text-white focus:outline-none focus:ring-2
focus:ring-indigo-500 dark:focus:ring-purple-500 placeholder-slate-400
dark:placeholder-slate-500 transition-colors" placeholder="0"
oninput="handleInput(${id})">
        </div>
        <div>
            <label class="block text-sm font-semibold text-slate-600 dark:text-
indigo-300 mb-1">Base System</label>
            <select id="inBase-${id}" class="w-full bg-white dark:bg-slate-900
border border-slate-300 dark:border-indigo-700 p-3 rounded-lg text-lg text-slate-
900 dark:text-white focus:outline-none focus:ring-2 focus:ring-indigo-500
dark:focus:ring-purple-500 transition-colors" onchange="handleInput(${id})">
                <option value="2">Binary (Base 2)</option>
                <option value="8">Octal (Base 8)</option>
                <option value="10" selected>Decimal (Base 10)</option>
                <option value="16">Hexadecimal (Base 16)</option>
            </select>
        </div>
    </div>

    <div id="out-${id}" class="mt-4 p-4 bg-white dark:bg-slate-950 border
border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] flex items-center
justify-center text-slate-500 dark:text-indigo-400 transition-colors duration-
300">
        Awaiting input...
    </div>
</div>
`
;
}

```

```

function reindexLabels() {
  const cases = document.querySelectorAll('[id^="case-"]');
  cases.forEach((caseEl, index) => {
    const varLetter = String.fromCharCode(65 + index);
    caseEl.setAttribute('data-var', varLetter);
    const labelEl = caseEl.querySelector('.case-label');
    if (labelEl) {
      labelEl.textContent = `Input ${varLetter}`;
    }
  });
}

function addCase() {
  caseCounter++;
  const container = document.getElementById('cases-container');
  container.insertAdjacentHTML('beforeend', createCaseHTML(caseCounter));
  reindexLabels();
  updateRemoveButtons();
  calculateTotal();
}

function removeCase(id) {
  const caseEl = document.getElementById(`case-${id}`);
  if (caseEl && document.querySelectorAll('[id^="case-"]').length > 2) {
    caseEl.remove();
    reindexLabels();
    updateRemoveButtons();
    calculateTotal();
  }
}

function updateRemoveButtons() {
  const buttons = document.querySelectorAll('.remove-btn');
  if (buttons.length <= 2) {
    buttons.forEach(btn => btn.classList.add('hidden'));
  } else {
    buttons.forEach(btn => btn.classList.remove('hidden'));
  }
}

function handleInput(id) {
  convertIndividualNumber(id);
  calculateTotal();
}

function convertIndividualNumber(id) {
  const rawValue = document.getElementById(`val-${id}`).value.trim();
  const inBase = parseInt(document.getElementById(`inBase-${id}`).value);
  const outputDiv = document.getElementById(`out-${id}`);

  if (rawValue === "") {
    outputDiv.innerHTML = "Awaiting input...";
    outputDiv.className = "mt-4 p-4 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] flex items-center

```

```

justify-center text-slate-500 dark:text-indigo-400 text-lg transition-colors";
    return;
  }

  let isValid = false;
  if (inBase === 2) isValid = /^-?[01]+(\.[01]+)?$/ .test(rawValue);
  if (inBase === 8) isValid = /^-?[0-7]+(\.[0-7]+)?$/ .test(rawValue);
  if (inBase === 10) isValid = /^-?[0-9]+(\.[0-9]+)?$/ .test(rawValue);
  if (inBase === 16) isValid = /^-?[0-9a-fA-F]+(\.[0-9a-fA-F]+)?$/ .test(rawValue);

  if (!isValid) {
    outputDiv.innerHTML = "<span class='text-red-600 dark:text-red-400 font-bold'>Invalid input for selected base.</span>";
    outputDiv.className = "mt-4 p-4 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] flex items-center justify-center transition-colors";
    return;
  }

  let decValue = parseToDecimal(rawValue, inBase);
  let comps = getComplements(decValue);

  const binStr = formatBase(decValue, 2);
  const octStr = formatBase(decValue, 8);
  const decStr = formatBase(decValue, 10);
  const hexStr = formatBase(decValue, 16);

  outputDiv.innerHTML = `
    <div class="grid grid-cols-2 lg:grid-cols-3 xl:grid-cols-6 gap-3 w-full">
      <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
        <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Binary</span>
        <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${binStr}</span></div>
      </div>
      <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
        <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Octal</span>
        <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${octStr}</span></div>
      </div>
      <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
        <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Decimal</span>
        <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${decStr}</span></div>
      </div>
      <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
        <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400

```

```

mb-1">Hexadecimal</span>
    <div class="overflow-x-auto"><span class="text-sm font-mono font-bold
text-indigo-700 dark:text-purple-300 break-all">${hexStr}</span></div>
    </div>
    <div class="bg-indigo-50 dark:bg-indigo-950/40 p-3 rounded-lg border
border-indigo-200 dark:border-indigo-800 min-w-0 flex flex-col">
    <span class="text-xs font-semibold text-indigo-600 dark:text-indigo-
400 mb-1">1's Comp (Int)</span>
    <div class="overflow-x-auto"><span class="text-sm font-mono font-bold
text-indigo-800 dark:text-purple-200 break-all">${comps.ones}</span></div>
    </div>
    <div class="bg-indigo-50 dark:bg-indigo-950/40 p-3 rounded-lg border
border-indigo-200 dark:border-indigo-800 min-w-0 flex flex-col">
    <span class="text-xs font-semibold text-indigo-600 dark:text-indigo-
400 mb-1">2's Comp (Int)</span>
    <div class="overflow-x-auto"><span class="text-sm font-mono font-bold
text-indigo-800 dark:text-purple-200 break-all">${comps.twos}</span></div>
    </div>
</div>
`;
    outputDiv.className = "mt-4 p-3 bg-white dark:bg-slate-950 border border-
slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] transition-colors";
}

function evaluatePostfix(postfix, variables) {
    const valStack = [];
    for (let token of postfix) {
        if (typeof token === 'number') {
            valStack.push(token);
        } else if (variables.hasOwnProperty(token)) {
            valStack.push(variables[token]);
        } else {
            if (valStack.length < 2) throw new Error("Invalid expression syntax.");
            const b = valStack.pop();
            const a = valStack.pop();
            if (token === '+') valStack.push(a + b);
            else if (token === '-') valStack.push(a - b);
            else if (token === '*') valStack.push(a * b);
            else if (token === '/') {
                if (b === 0) throw new Error("Division by zero error.");
                valStack.push(a / b);
            }
        }
    }
    if (valStack.length !== 1) throw new Error("Invalid expression syntax.");
    return valStack[0];
}

function calculateTotal() {
    const cases = document.querySelectorAll('[id^="case-"]');
    const totalDiv = document.getElementById('final-total-container');
    const exprString = document.getElementById('math-
expression').value.trim().toUpperCase();

```

```

window.globalVars = {};
let displayStrs = {};
let allValid = true;

cases.forEach(caseEl => {
  const id = caseEl.id.split('-')[1];
  const varName = caseEl.getAttribute('data-var');
  const rawValue = document.getElementById(`val-${id}`).value.trim();
  const inBase = parseInt(document.getElementById(`inBase-${id}`).value);

  if (!rawValue) {
    allValid = false;
    return;
  }

  let isValid = false;
  if (inBase === 2) isValid = /^-[01]+(\.[01]+)?$/ .test(rawValue);
  if (inBase === 8) isValid = /^-[0-7]+(\.[0-7]+)?$/ .test(rawValue);
  if (inBase === 10) isValid = /^-[0-9]+(\.[0-9]+)?$/ .test(rawValue);
  if (inBase === 16) isValid = /^-[0-9a-fA-F]+(\.[0-9a-fA-F]+)?$/ .test(rawValue);

  if (!isValid) {
    allValid = false;
    return;
  }

  let decValue = parseToDecimal(rawValue, inBase);
  window.globalVars[varName] = decValue;

  const sub = inBase === 2 ? '2' : inBase === 8 ? '8' : inBase === 10 ? '10' : '16';
  displayStrs[varName] = `(${rawValue})${sub}`;
});

updateComplementSelects();

if (!allValid || Object.keys(window.globalVars).length === 0) {
  totalDiv.innerHTML = "<p class='text-slate-400 text-lg'>Enter valid numbers in all active fields to compute.</p>";
  return;
}

if (!exprString) {
  totalDiv.innerHTML = "<p class='text-slate-400 text-lg'>Please enter a mathematical expression.</p>";
  return;
}

const tokens = exprString.match(/[A-Z]+|[0-9]*\.[0-9]+|[+\-*/()]/g);
if (!tokens) {
  totalDiv.innerHTML = "<p class='text-red-400 text-xl font-bold'>Error: Invalid characters in expression.</p>";
  return;
}

```

```

}

const precedence = { '+': 1, '-': 1, '*': 2, '/': 2 };
const postfix = [];
const opStack = [];
let formattedEqTokens = [];

try {
  for (let token of tokens) {
    if (window.globalVars.hasOwnProperty(token)) {
      postfix.push(token);
      formattedEqTokens.push(`${displayStrs[token]}
</span>`);
    } else if (/^[0-9]*\.[0-9]+$/.test(token)) {
      postfix.push(Number(token));
      formattedEqTokens.push(`${token}</span>`);
    } else if (['+', '-', '*', '/'].includes(token)) {
      let symbol = token === '*' ? 'x' : token === '/' ? '÷' : token;
      formattedEqTokens.push(`${symbol}
</span>`);
      while (opStack.length > 0 && opStack[opStack.length - 1] !== '(' &&
        precedence[opStack[opStack.length - 1]] >= precedence[token]) {
        postfix.push(opStack.pop());
      }
      opStack.push(token);
    } else if (token === '(') {
      formattedEqTokens.push(`</span>`);
      opStack.push(token);
    } else if (token === ')') {
      formattedEqTokens.push(`></span>`);
      while (opStack.length > 0 && opStack[opStack.length - 1] !== '(') {
        postfix.push(opStack.pop());
      }
      if (opStack.length === 0) throw new Error("Mismatched parentheses.");
      opStack.pop();
    } else {
      throw new Error(`Unknown variable: ${token}`);
    }
  }
  while (opStack.length > 0) {
    const op = opStack.pop();
    if (op === '(' || op === ')') throw new Error("Mismatched
parentheses.");
    postfix.push(op);
  }

  const finalResult = evaluatePostfix(postfix, window.globalVars);
  const comps = getComplements(finalResult);

  const binStr = formatBase(finalResult, 2);
  const octStr = formatBase(finalResult, 8);
  const decStr = formatBase(finalResult, 10);
  const hexStr = formatBase(finalResult, 16);

```

```

totalDiv.innerHTML = `
    <div class="mb-6 p-4 bg-slate-800 rounded-lg border border-slate-700
shadow-inner overflow-x-auto">
        <p class="text-indigo-300 text-sm font-bold uppercase tracking-wider
mb-2">Equation Breakdown</p>
        <p class="text-xl md:text-2xl font-mono whitespace-
nowrap">${formattedEqTokens.join('')} <span class="text-pink-400 mx-2">=</span>
</p>
    </div>

    <div class="grid grid-cols-2 lg:grid-cols-3 xl:grid-cols-6 gap-4">
        <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-
left overflow-hidden">
            <span class="block text-sm font-bold text-slate-400 mb-
1">Binary</span>
            <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-
mono font-bold text-green-400 break-all">${binStr}</span></div>
        </div>
        <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-
left overflow-hidden">
            <span class="block text-sm font-bold text-slate-400 mb-
1">Octal</span>
            <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-
mono font-bold text-blue-400 break-all">${octStr}</span></div>
        </div>
        <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-
left overflow-hidden">
            <span class="block text-sm font-bold text-slate-400 mb-
1">Decimal</span>
            <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-
mono font-bold text-yellow-400 break-all">${decStr}</span></div>
        </div>
        <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-
left overflow-hidden">
            <span class="block text-sm font-bold text-slate-400 mb-
1">Hexadecimal</span>
            <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-
mono font-bold text-pink-400 break-all">${hexStr}</span></div>
        </div>
        <div class="bg-slate-800 p-4 rounded-xl border border-indigo-500/50
text-left overflow-hidden">
            <span class="block text-sm font-bold text-indigo-300 mb-1">1's Comp
(Int)</span>
            <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-
mono font-bold text-indigo-400 break-all">${comps.ones}</span></div>
        </div>
        <div class="bg-slate-800 p-4 rounded-xl border border-indigo-500/50
text-left overflow-hidden">
            <span class="block text-sm font-bold text-indigo-300 mb-1">2's Comp
(Int)</span>
            <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-
mono font-bold text-indigo-400 break-all">${comps.twos}</span></div>
        </div>
    </div>
`

```

```

    `;
  } catch (err) {
    totalDiv.innerHTML = `
```

```

let mBin = toSignedBin(mVal, bits);
let sBinAbs = toSignedBin(sVal, bits);

let sOnes = sBinAbs.split('').map(b => b==='0'? '1': '0').join('');
let sTwos = addBinaryStr(sOnes, '1'.padStart(bits, '0')).sum;

let add1 = addBinaryStr(mBin, sOnes);
let finalOnes = add1.sum;
let endCarryTxt = "No carry, result is negative (in 1's complement form).";
if (add1.carry) {
  finalOnes = addBinaryStr(add1.sum, '1'.padStart(bits, '0')).sum;
  endCarryTxt = "Carry = 1. Add End-Around Carry (+1).";
}

let add2 = addBinaryStr(mBin, sTwos);
let finalTwos = add2.sum;

out.innerHTML = `
  <div class="grid grid-cols-1 md:grid-cols-2 gap-8 text-left mt-4">
    <div class="bg-slate-900 p-6 rounded-xl border border-slate-700 shadow-inner">
      <h3 class="text-teal-400 font-bold mb-4 border-b border-slate-700 pb-2">1's Complement Method</h3>
      <p class="text-slate-400 mb-1">M: <span class="text-white">${mBin}</span></p>
      <p class="text-slate-400 mb-1">S: <span class="text-white">${sBinAbs}</span></p>
      <p class="text-slate-400 mb-3 border-b border-slate-700 pb-2">1's Comp(S): <span class="text-pink-400">${sOnes}</span></p>
      <p class="text-slate-400 mb-1">Add M + 1's Comp(S):</p>
      <p class="text-white font-bold tracking-widest bg-slate-800 p-2 rounded mb-2">${mBin}<br>+${sOnes}<br>-----<br>${add1.carry ? '1' : '0'}${add1.sum}</p>
      <p class="text-emerald-400 text-sm mb-2">${endCarryTxt}</p>
      <p class="text-white font-bold tracking-widest bg-slate-800 p-2 rounded">Final: ${finalOnes}</p>
    </div>

    <div class="bg-slate-900 p-6 rounded-xl border border-slate-700 shadow-inner">
      <h3 class="text-teal-400 font-bold mb-4 border-b border-slate-700 pb-2">2's Complement Method</h3>
      <p class="text-slate-400 mb-1">M: <span class="text-white">${mBin}</span></p>
      <p class="text-slate-400 mb-1">S: <span class="text-white">${sBinAbs}</span></p>
      <p class="text-slate-400 mb-3 border-b border-slate-700 pb-2">2's Comp(S): <span class="text-pink-400">${sTwos}</span></p>
      <p class="text-slate-400 mb-1">Add M + 2's Comp(S):</p>
      <p class="text-white font-bold tracking-widest bg-slate-800 p-2 rounded mb-2">${mBin}<br>+${sTwos}<br>-----<br>${add2.carry ? '1' : '0'}${add2.sum}</p>
      <p class="text-emerald-400 text-sm mb-2">Discard carry out.</p>
    </div>
  </div>

```

```
    <p class="text-white font-bold tracking-widest bg-slate-800 p-2
rounded">Final: ${finalTwos}</p>
  </div>
</div>
`
;
}

addCase();
addCase();
addCase();
</script>

</body>
</html>
```

Test Cases

Test Case 1: Binary Conversion

The screenshot shows a web application interface for base conversion. It features a dark theme with purple accents. At the top, there's a header 'Input 1' with a close button. Below it, there are two main sections: 'Value' and 'Base System'. The 'Value' section has a text input field containing '1010'. The 'Base System' section has a dropdown menu currently showing 'Binary (Base 2)'. Below these sections, there are four buttons representing different base systems: 'Binary', 'Octal', 'Decimal', and 'Hexadecimal'. Each button displays the converted value: '1010' for Binary, '12' for Octal, '10' for Decimal, and 'A' for Hexadecimal.

1. Input Value: 1010
2. Selected Base: Binary (Base 2)
3. Expected Output: Binary: 1010, Octal: 12, Decimal: 10, Hexadecimal: A

Test Case 2: Hexadecimal Conversion

The screenshot shows the same web application interface as the previous one, but for a hexadecimal conversion test case. The 'Value' input field now contains '1F'. The 'Base System' dropdown menu is now set to 'Hexadecimal (Base 16)'. The output buttons show the following converted values: '11111' for Binary, '37' for Octal, '31' for Decimal, and '1F' for Hexadecimal.

1. Input Value: 1F
2. Selected Base: Hexadecimal (Base 16)
3. Expected Output: Binary: 11111, Octal: 37, Decimal: 31, Hexadecimal: 1F

Test Case 3: Invalid Input Handling

Input 1

Value

Base System

9

Octal (Base 8)

Invalid input for selected base.

1. Input Value: 9
2. Selected Base: Octal (Base 8)
3. Expected Output: "Invalid input for selected base." text appears in red.

Test Case 4: Mathematical Bounds Limits

Input 1

Value

99999999999999999999

Base System

Decimal (Base 10)

Binary

10101101011111000111010111
10001011010110001100001
111111111111111111

Octal

1265707274265430377777
7

Decimal

99999999999999999999

Hexadecimal

56BC75E2D630FFFFFF

1. Input Value: 9999999999999999999
2. Selected Base: Decimal (Base 10)
3. Expected Output: Hexadecimal correctly outputs 56BC75E2D63100000 instead of converting the string into scientific notation.

Test Case 5: Operator Precedence (Binary + Octal + Decimal)

The screenshot shows the 'System Converter & Calculator' application. The 'Arithmetic Expression' section displays $A + B * C$. Below it, three input sections are visible: 'Input A' with value 1010 (Base 2), 'Input B' with value 20 (Base 8), and 'Input C' with value 2 (Base 10). The 'Final Arithmetic Result' section shows the equation breakdown: $(1010)_2 + (20)_8 \times (2)_{10} =$. The results are displayed in four boxes: Binary 101010, Octal 52, Decimal 42, and Hexadecimal 2A.

System Converter & Calculator

Arithmetic Expression

Use variables (A, B, C, ...) and operators (+, -, *, /, parentheses). Example: $(A + B - C) * D$

$A + B * C$

Input A

Value: 1010 Base System: Binary (Base 2)

Binary: 1010 Octal: 12 Decimal: 10 Hexadecimal: A

Input B

Value: 20 Base System: Octal (Base 8)

Binary: 10000 Octal: 20 Decimal: 16 Hexadecimal: 10

Input C

Value: 2 Base System: Decimal (Base 10)

Binary: 10 Octal: 2 Decimal: 2 Hexadecimal: 2

Final Arithmetic Result

EQUATION BREAKDOWN

$(1010)_2 + (20)_8 \times (2)_{10} =$

Binary: 101010 Octal: 52

Decimal: 42 Hexadecimal: 2A

1. Inputs: Input A = 1010 (Base 2), Input B = 20 (Base 8), Input C = 2 (Base 10)
2. Arithmetic Expression: $A + B * C$
3. Expected Equation Breakdown: $(1010)_2 + (20)_8 \times (2)_{10}$
4. Expected Logic: Multiplication executes before addition. $10 + (16 \times 2) = 42$.
5. Final Arithmetic Result: Binary: 101010, Octal: 52, Decimal: 42, Hexadecimal: 2A.

Test Case 6: Parenthetical Grouping (Binary + Decimal + Hexadecimal)

System Converter & Calculator

Arithmetic Expression
Use variables (A, B, C...) and operators (+, -, *, /, parentheses). Example: (A + B) * C

(A - B) / C

Input A
Value: 10000 Base System: Binary (Base 2)
Binary: 10000 Octal: 20 Decimal: 10 Hexadecimal: 10

Input B
Value: 4 Base System: Decimal (Base 10)
Binary: 100 Octal: 4 Decimal: 4 Hexadecimal: 4

Input C
Value: 2 Base System: Hexadecimal (Base 16)
Binary: 10 Octal: 2 Decimal: 2 Hexadecimal: 2

Final Arithmetic Result
EQUATION BREAKDOWN
 $((10000)_2 - (4)_{10}) \div (2)_{16} =$
Binary: 110 Octal: 6
Decimal: 6 Hexadecimal: 6

1. Inputs: Input A = 10000 (Base 2), Input B = 4 (Base 10), Input C = 2 (Base 16)
2. Arithmetic Expression: (A - B) / C
3. Expected Equation Breakdown: $((10000)_2 - (4)_{10}) \div (2)_{16}$
4. Expected Logic: Parentheses force subtraction before division. $(16 - 4) \div 2 = 6$.
5. Final Arithmetic Result: Binary: 110, Octal: 6, Decimal: 6, Hexadecimal: 6.

Test Case 7: Mixed Operators (Octal + Decimal + Hexadecimal)

The screenshot shows the 'System Converter & Calculator' application interface. At the top, there is an 'Arithmetic Expression' field containing $A * B - C$. Below this, there are three input sections: 'Input A', 'Input B', and 'Input C'. 'Input A' has a value of 12 and is set to Octal (Base 8). 'Input B' has a value of 5 and is set to Decimal (Base 10). 'Input C' has a value of F and is set to Hexadecimal (Base 16). Each input section also displays its value in other bases: Binary, Octal, Decimal, and Hexadecimal. The interface is dark-themed with purple accents.

System Converter & Calculator

Arithmetic Expression
Use variables (A, B, C...) and operators (+, -, *, /, parentheses). Example: $(A + B - C) * D$

$A * B - C$

Input A
Value: 12 Base System: Octal (Base 8)
Binary: 1010 Octal: 12 Decimal: 10 Hexadecimal: A

Input B
Value: 5 Base System: Decimal (Base 10)
Binary: 101 Octal: 5 Decimal: 5 Hexadecimal: 5

Input C
Value: F Base System: Hexadecimal (Base 16)
Binary: 1111 Octal: 17 Decimal: 15 Hexadecimal: F

+ Add Input Number

Final Arithmetic Result

EQUATION BREAKDOWN
 $(12)_8 * (5)_{10} - (F)_{16} =$

Binary: 100011 Octal: 43
Decimal: 35 Hexadecimal: 23

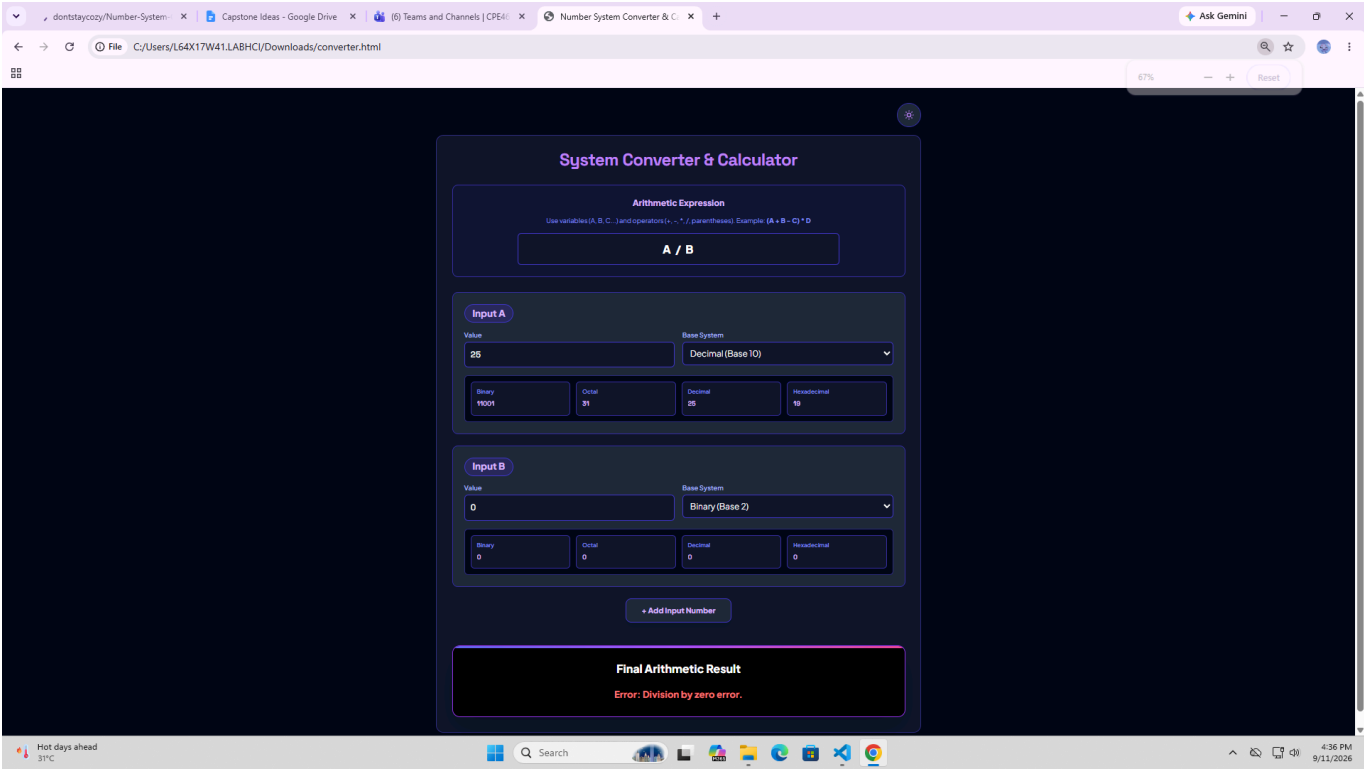
1. Inputs: Input A = 12 (Base 8), Input B = 5 (Base 10), Input C = F (Base 16)
2. Arithmetic Expression: $A * B - C$
3. Expected Equation Breakdown: $(12)_8 * (5)_{10} - (F)_{16}$
4. Expected Logic: Multiplication executes before subtraction. $(10 * 5) - 15 = 35$.
5. Final Arithmetic Result: Binary: 100011, Octal: 43, Decimal: 35, Hexadecimal: 23.

Test Case 8: Complex Multi-Variable Expression (Binary + Octal + Hexadecimal + Binary)

The screenshot shows a web application titled "System Converter & Calculator". It features an "Arithmetic Expression" field containing $(A + B - C) * D$. Below this are three input sections: "Input A" with value "1100" and base "Binary (Base 2)", "Input B" with value "10" and base "Octal (Base 8)", and "Input C" with value "A" and base "Hexadecimal (Base 16)". At the bottom, the "Final Arithmetic Result" section displays the equation breakdown: $((1100)_2 + (10)_8 - (A)_{16}) \times (11)_2 =$, and the final result in four bases: Binary (11110), Octal (36), Decimal (30), and Hexadecimal (1E).

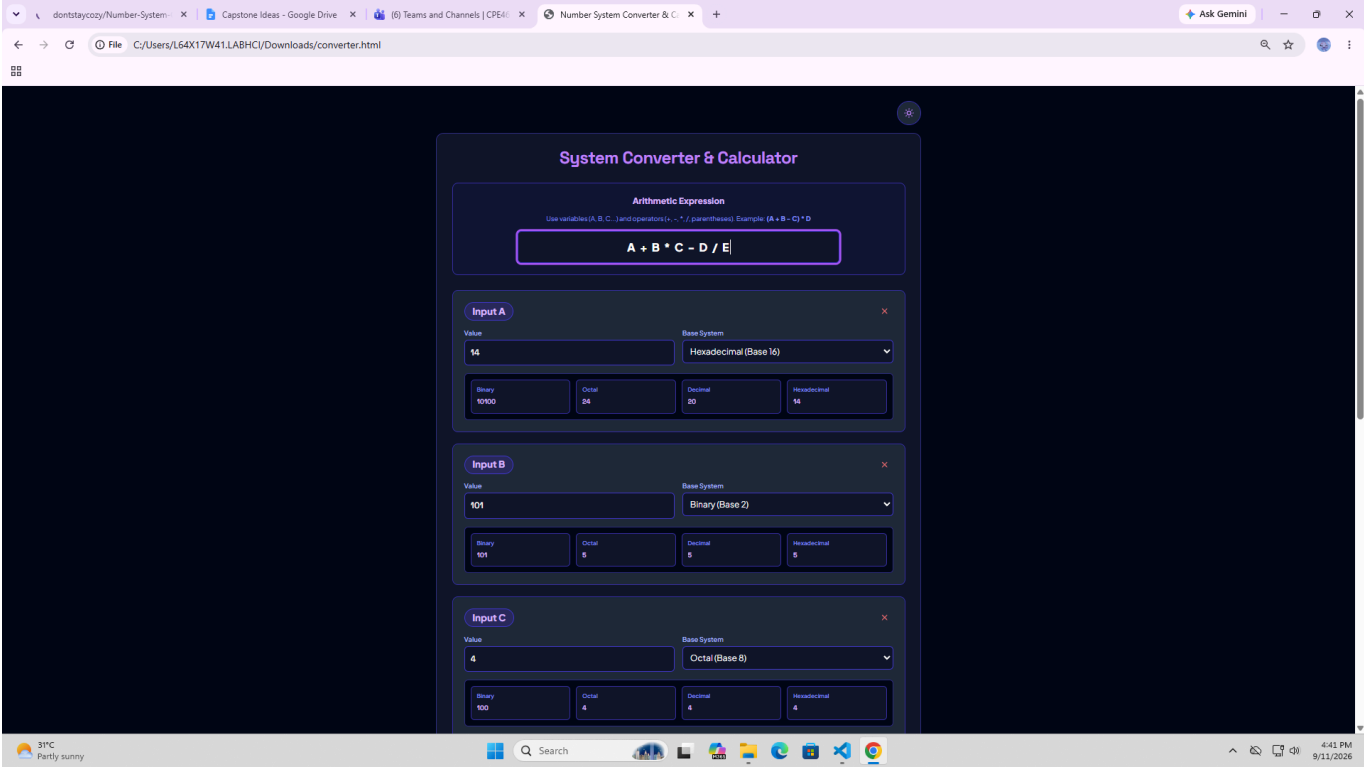
1. Inputs: Input A = 1100 (Base 2), Input B = 10 (Base 8), Input C = A (Base 16), Input D = 11 (Base 2)
2. Arithmetic Expression: $(A + B - C) * D$
3. Expected Equation Breakdown: $((1100)_2 + (10)_8 - (A)_{16}) \times (11)_2$
4. Expected Logic: Left-to-right inside parentheses, followed by multiplication. $(12 + 8 - 10) \times 3 = 30$.
5. Final Arithmetic Result: Binary: 11110, Octal: 36, Decimal: 30, Hexadecimal: 1E.

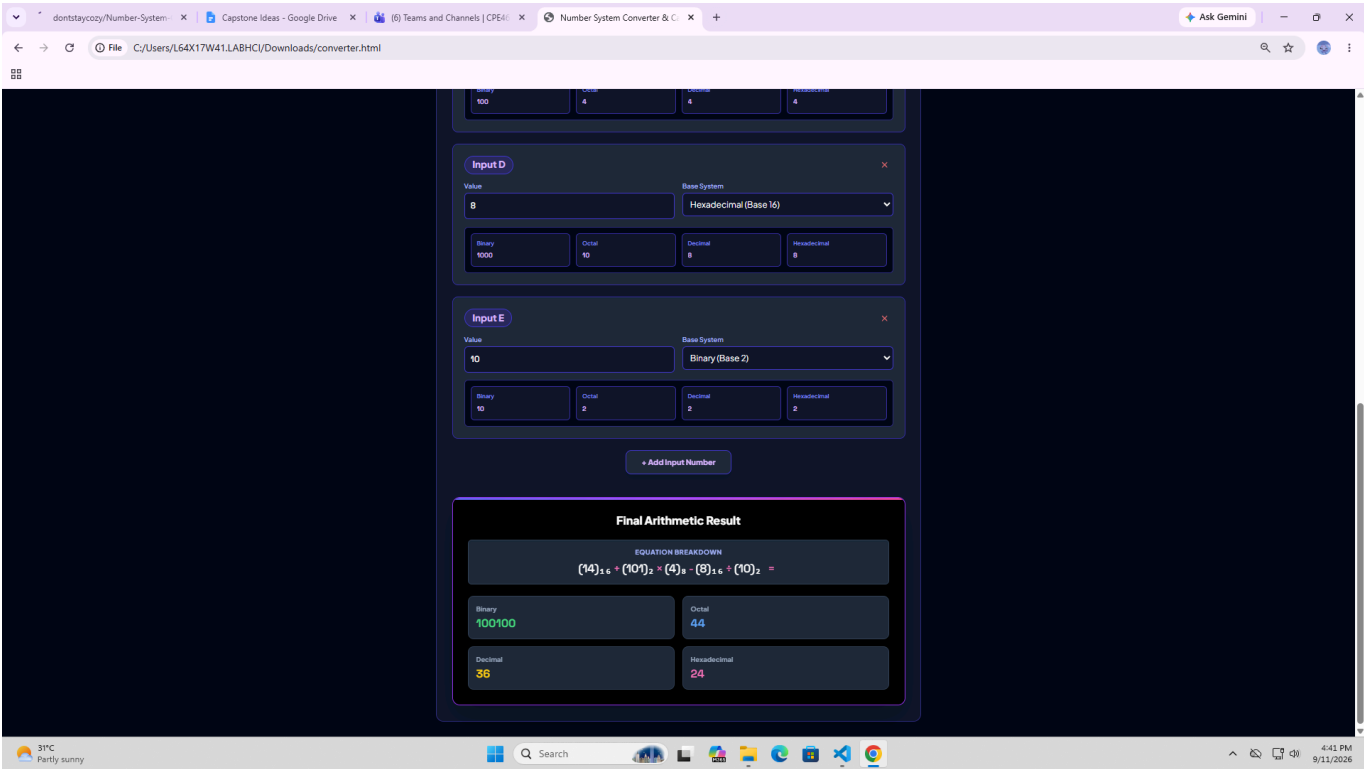
Test Case 9: Arithmetic Error Handling (Division by Zero)



- 1. Inputs: **Input A** = 25 (Base 10), **Input B** = 0 (Base 2)
- 2. Arithmetic Expression: **A / B**
- 3. Expected Equation Breakdown: None.
- 4. Expected Output: Red error message stating "Error: Division by zero error." prevents the calculation.

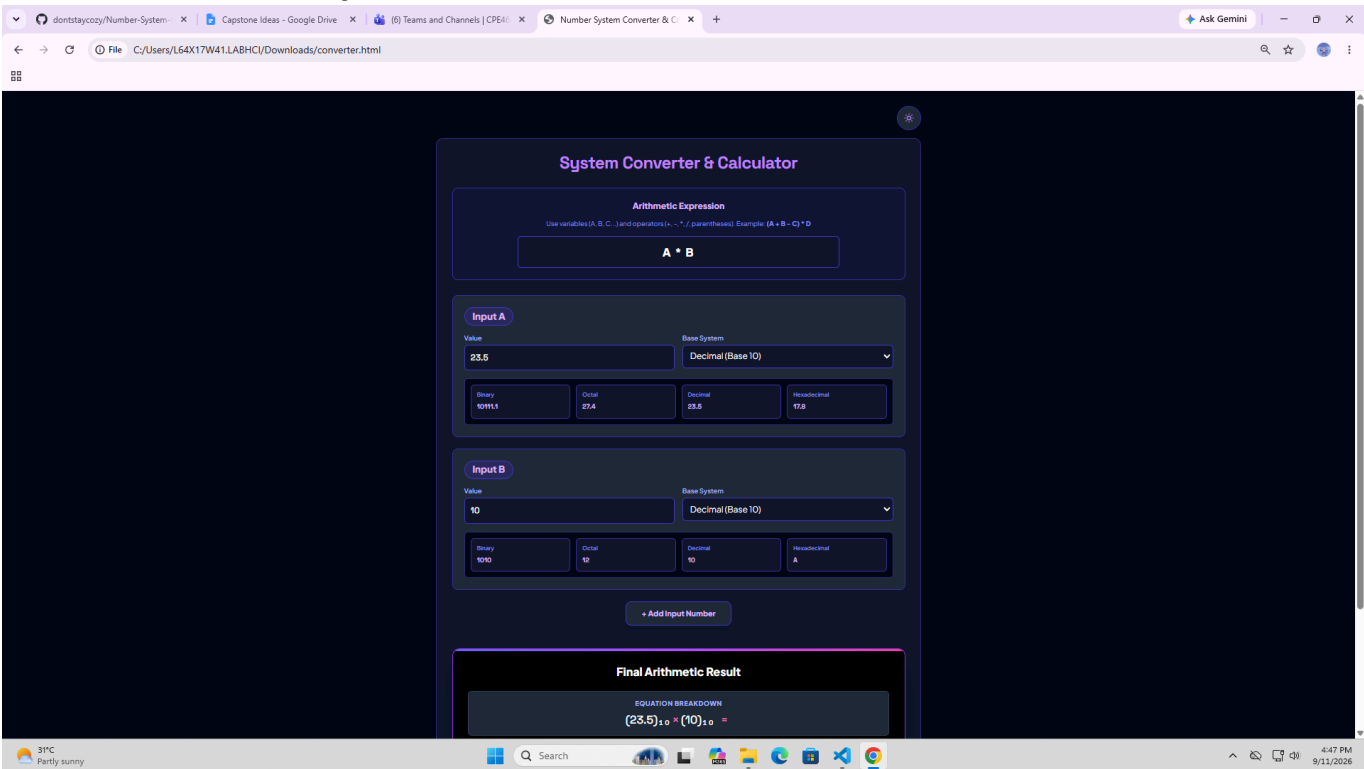
Test Case 10: Comprehensive Order of Operations (Mixed Bases)

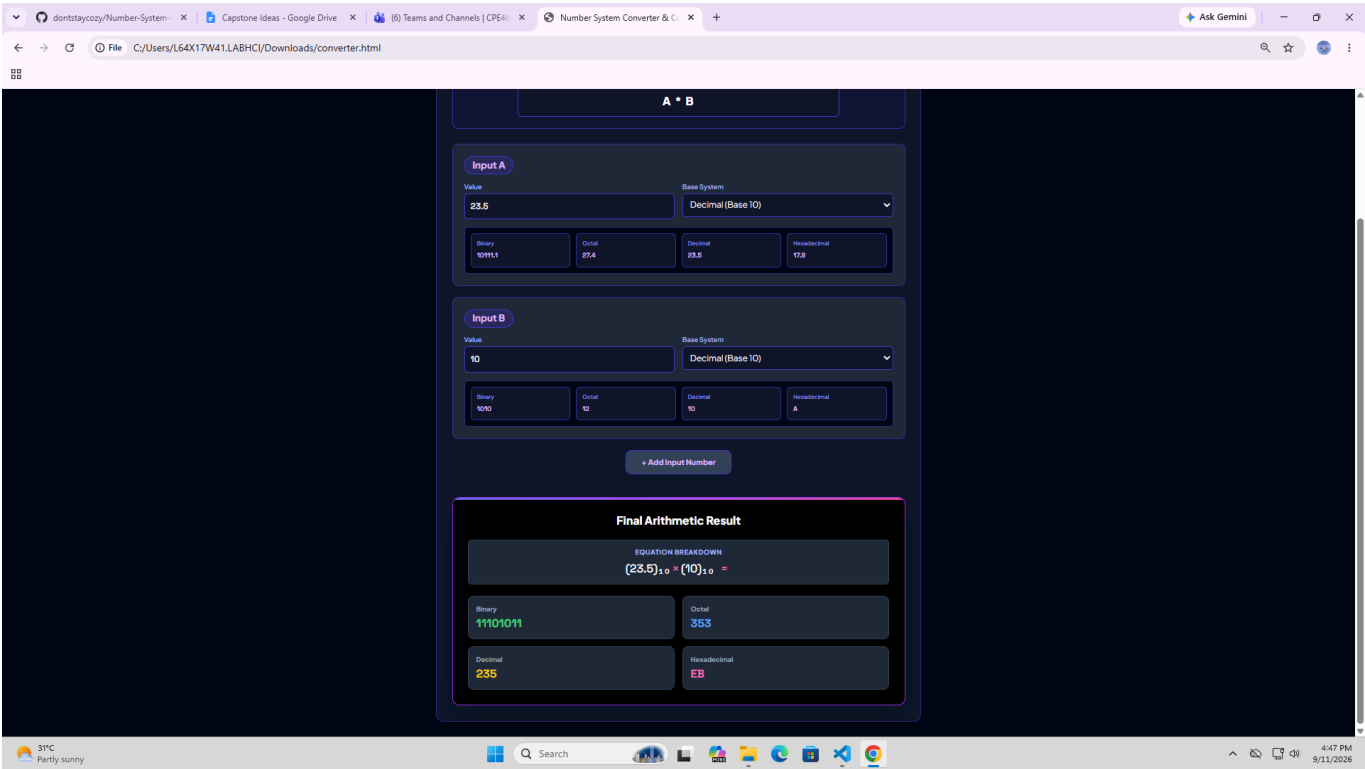




- 1. Inputs: Input A = 14 (Hexadecimal), Input B = 101 (Binary), Input C = 4 (Octal), Input D = 8 (Hexadecimal), Input E = 10 (Binary)
- 2. Arithmetic Expression: $A + B * C - D / E$
- 3. Expected Equation Breakdown: $(14)_{16} + (101)_2 \times (4)_8 - (8)_{16} \div (10)_2$
- 4. Expected Logic: Base conversions equate to $(20 + 5 \times 4 - 8 \div 2)$. Multiplication and division execute first $(5 \times 4 = 20)$ and $(8 \div 2 = 4)$, followed by addition and subtraction $(20 + 20 - 4) = 36$.
- 5. Final Arithmetic Result: Binary: 100100, Octal: 44, Decimal: 36, Hexadecimal: 24.

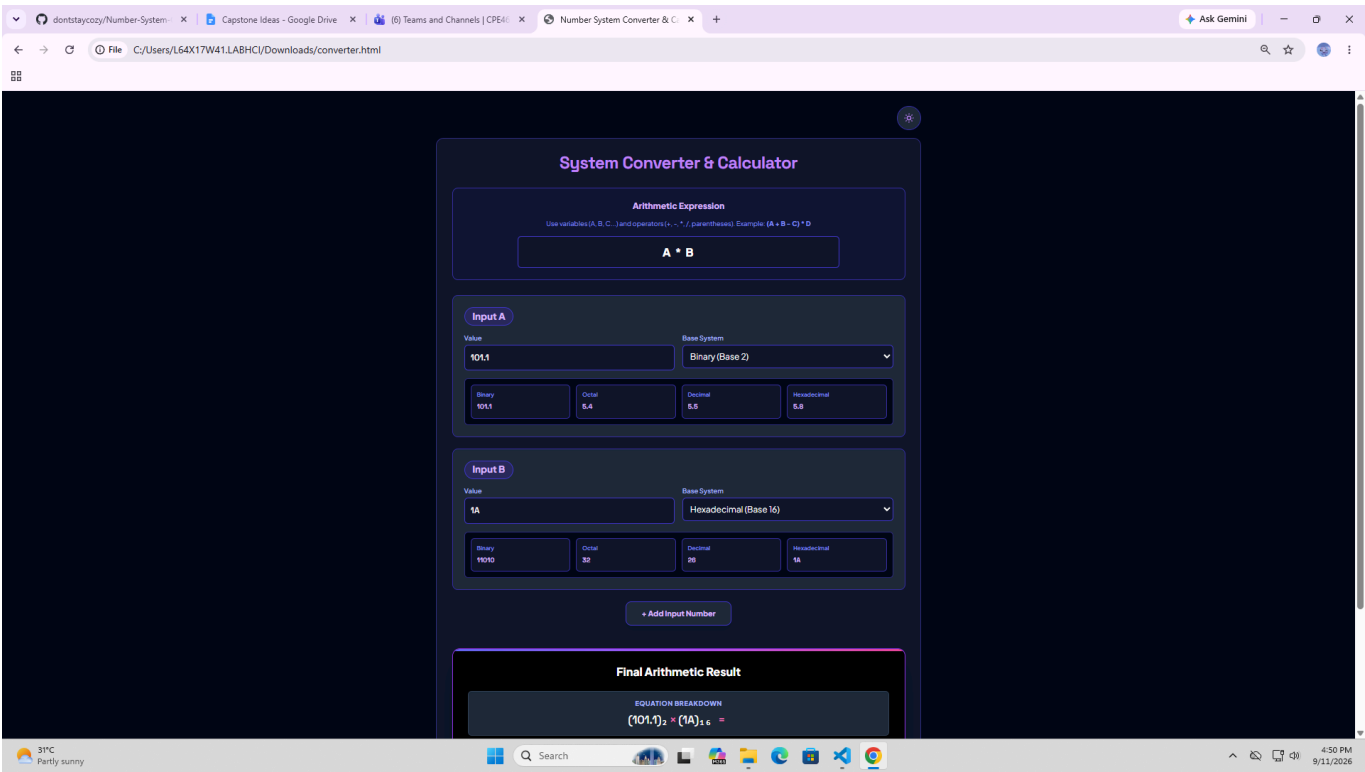
Test Case 11: Fractional Input Calculation

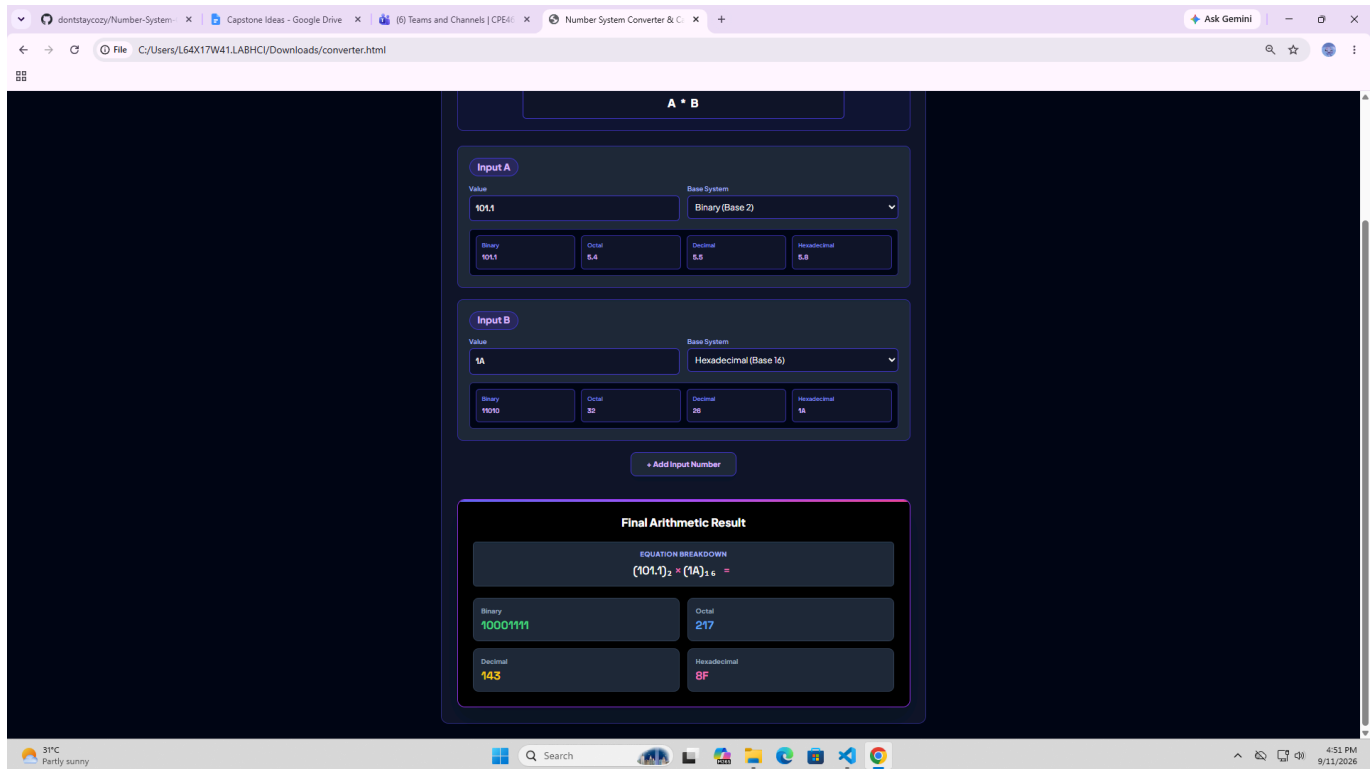




- 1. Inputs: Input A = 23.5 (Decimal), Input B = 10 (Decimal)
- 2. Arithmetic Expression: A * B
- 3. Expected Equation Breakdown: $(23.5)_{10} \times (10)_{10}$
- 4. Expected Logic: Multiplication of a fractional decimal by a whole decimal. $23.5 \times 10 = 235$.
- 5. Final Arithmetic Result: Binary: 11101011, Octal: 353, Decimal: 235, Hexadecimal: EB.

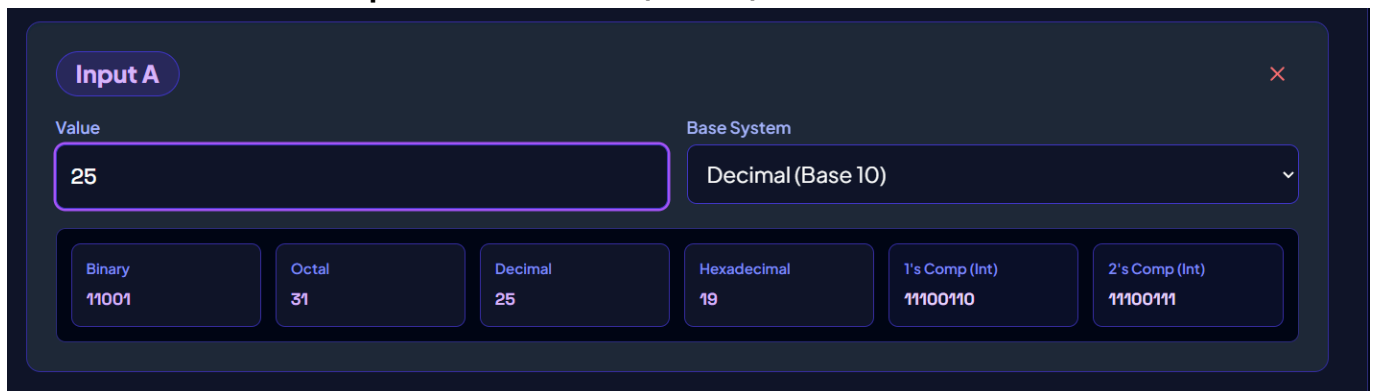
Test Case 12: Non-Decimal Fractional Input Calculation





1. Inputs: **Input A** = 101.1 (Binary), **Input B** = 1A (Hexadecimal)
2. Arithmetic Expression: $A + B$
3. Expected Equation Breakdown: $(101.1)_2 + (1A)_{16}$
4. Expected Logic: Base conversions equate to $(5.5 + 26)$. The fractional binary successfully adds to the whole hexadecimal. $5.5 + 26 = 31.5$.
5. Final Arithmetic Result: Binary: 11111.1, Octal: 37.4, Decimal: 31.5, Hexadecimal: 1F.8.

Test Case 13: Standard Complement Generation (Base 10)



1. Inputs: **Input A** = 25 (Decimal)
2. Expected Logic: The system computes standard base conversions and dynamically generates the 8-bit padded 1's and 2's complements for the positive integer.
3. Expected Output: Binary: 11001, Octal: 31, Decimal: 25, Hexadecimal: 19, 1's Comp (Int): 11100110, 2's Comp (Int): 11100111.

Test Case 14: Complement Subtraction (Positive Result / Minuend > Subtrahend)

The application interface shows the following inputs and outputs:

Input A: Value: 15, Base System: Decimal (Base 10).
 Binary: 1111, Octal: 17, Decimal: 15, Hexadecimal: F, 1's Comp (Int): 11110000, 2's Comp (Int): 11110001.

Input B: Value: 5, Base System: Decimal (Base 10).
 Binary: 101, Octal: 5, Decimal: 5, Hexadecimal: 5, 1's Comp (Int): 11111010, 2's Comp (Int): 11111011.

Complement Subtraction Method: Select two variables to demonstrate step-by-step subtraction (Minuend - Subtrahend) using complements. Uses absolute integer parts.

1's Complement Method:
 M: 00001111
 S: 00000101
 1's Comp(S): 11111010
 Add M + 1's Comp(S):

```

00001111
+11111010
-----
1 00001001

```

 Carry = 1. Add End-Around Carry (+1).
 Final: 00001010

2's Complement Method:
 M: 00001111
 S: 00000101
 2's Comp(S): 11111011
 Add M + 2's Comp(S):

```

00001111
+11111011
-----
1 00001010

```

 Discard carry out.
 Final: 00001010

1. Inputs: Set Minuend dropdown to Variable A (Decimal 15), Set Subtrahend to Variable B (Decimal 5).
2. Expected Logic: The tool processes 15 - 5 using 8-bit representation.
3. 1's Complement Output: Converts S(5) to 00000101, 1's comp 11111010. Adds to M(15) 00001111. Yields carry=1. Adds end-around carry to yield 00001010 (10).
4. 2's Complement Output: Converts S(5) to 2's comp 11111011. Adds to M(15). Yields carry=1 (discarded), resulting directly in 00001010 (10).

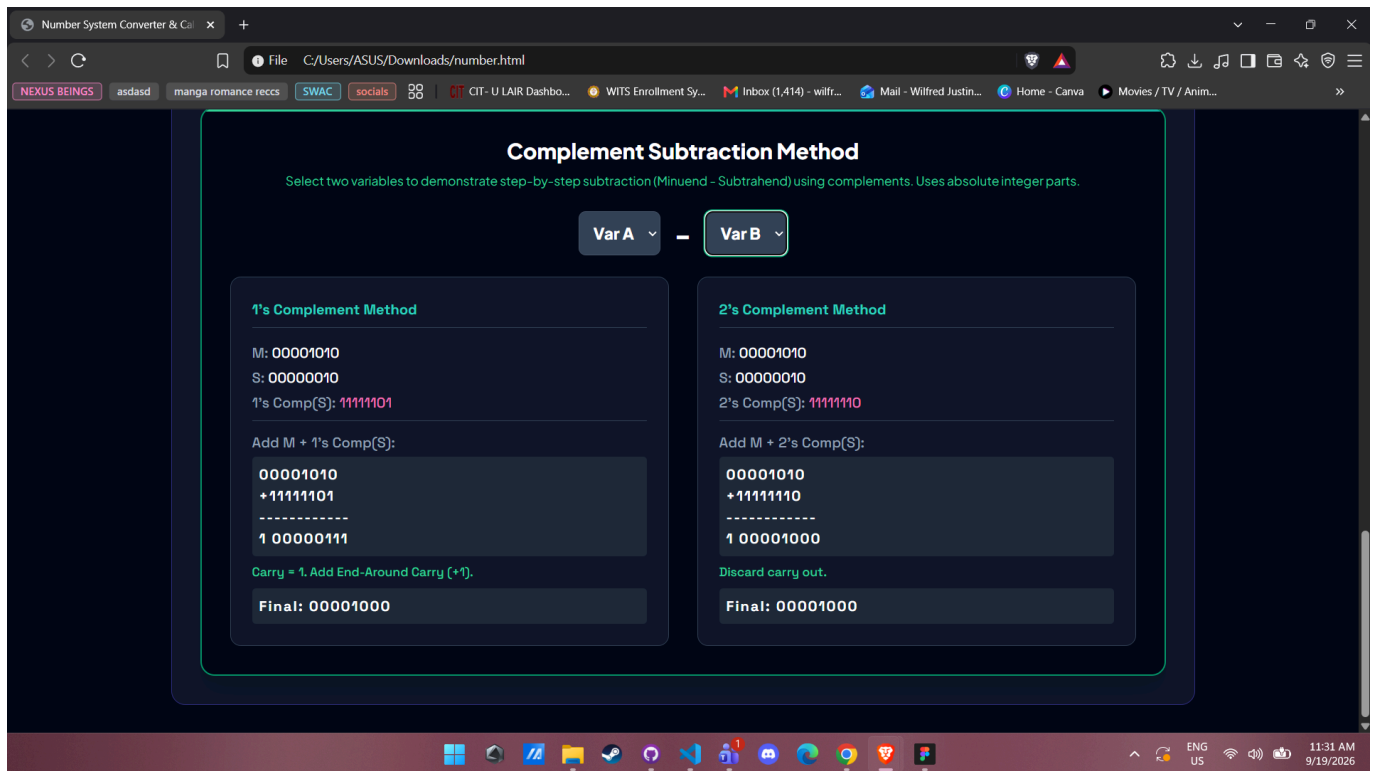
Test Case 15: Complement Subtraction (Negative Result / Minuend < Subtrahend)

The screenshot shows the 'Number System Converter & Calculator' application. It has two input sections, 'Input A' and 'Input B'.
Input A: Value is 5, Base System is Decimal (Base 10).
Input B: Value is 15, Base System is Decimal (Base 10).
Below each input, there are buttons for different number systems: Binary, Octal, Decimal, Hexadecimal, 1's Comp (Int), and 2's Comp (Int).
For Input A, the outputs are: Binary 101, Octal 5, Decimal 5, Hexadecimal 5, 1's Comp (Int) 1111010, 2's Comp (Int) 1111011.
For Input B, the outputs are: Binary 1111, Octal 17, Decimal 15, Hexadecimal F, 1's Comp (Int) 11110000, 2's Comp (Int) 11110001.

1. Inputs: Set Minuend dropdown to Variable A (Decimal 5), Set Subtrahend to Variable B (Decimal 15).
2. Expected Logic: The tool processes $5 - 15$ using 8-bit representation.
3. 1's Complement Output: Converts S(15) to 00001111, 1's comp 11110000. Adds to M(5) 00000101. Yields no carry. Final result is a negative number in 1's complement form: 11110101.
4. 2's Complement Output: Converts S(15) to 2's comp 11110001. Adds to M(5). Yields no carry. Final result is a negative number in 2's complement form: 11110110.

Test Case 16: Final Arithmetic Result with Complements

The screenshot shows the 'Number System Converter & Calculator' application. It has two input sections, 'Input A' and 'Input B'.
Input A: Value is 10, Base System is Decimal (Base 10).
Input B: Value is 2, Base System is Decimal (Base 10).
Below each input, there are buttons for different number systems: Binary, Octal, Decimal, Hexadecimal, 1's Comp (Int), and 2's Comp (Int).
For Input A, the outputs are: Binary 1010, Octal 12, Decimal 10, Hexadecimal A, 1's Comp (Int) 11110101, 2's Comp (Int) 11110110.
For Input B, the outputs are: Binary 10, Octal 2, Decimal 2, Hexadecimal 2, 1's Comp (Int) 11111101, 2's Comp (Int) 11111110.
At the bottom, there is a button labeled '+ Add Input Number'.



1. Inputs: Input A = 10 (Decimal), Input B = 2 (Decimal)
2. Arithmetic Expression: $A - B$
3. Expected Equation Breakdown: $(10)_{10} - (2)_{10}$
4. Expected Logic: The system computes $10 - 2 = 8$, then dynamically generates the complements for the final computed result.
5. Final Arithmetic Result: Binary: 1000, Octal: 10, Decimal: 8, Hexadecimal: 8, 1's Comp (Int): 11110111, 2's Comp (Int): 11111000.

Code Screenshots

```
File Edit Selection View Go Run Terminal Help
Number-System-Converter > converter.html > html:dark > head > script > theme > extend
1 <!DOCTYPE html>
2 <html lang="en" class="dark">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Number System Converter & Calculator</title>
7
8 <link rel="preconnect" href="https://fonts.googleapis.com">
9 <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
10 <link href="https://fonts.googleapis.com/css2?family=PlusJakartaSans:wght@400;600;800&family=SpaceGrotesk:wght@500;700&display=swap" rel="stylesheet">
11
12 <script src="https://cdn.tailwindcss.com"></script>
13 <script>
14   tailwind.config = {
15     darkMode: 'class',
16     theme: {
17       extend: {
18         fontFamily: {
19           sans: ["Plus Jakarta Sans", 'sans-serif'],
20           mono: ["Space Grotesk", 'monospace'],
21         }
22       }
23     }
24   }
25 </script>
26 </head>
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
28
29 <div class="max-w-5xl mx-auto">
30   <div class="flex justify-end mb-4">
31     <button onclick="toggleTheme()" class="p-3 rounded-full shadow-lg bg-indigo-100 dark:bg-slate-800 text-indigo-700 dark:text-purple-400 hover:bg-indigo-200 dark:ho
32       <svg class="w-6 h-6 hidden dark:block" fill="none" stroke="currentColor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
33         <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M12 3v10m0 0h10m0 0H12M15 3.464 15.5 4 15 4.5 14.5 4 14 3.464 14 3 14.5 2.5 15 2 15.5 2.5 16 3 15.5 3.464 16 4 15.5 4.5 16 5 15.5 5.464 16 6 15.5 6.464 16 7 15.5 6.464 16 7.5 15.5 7.5 16 8 15.5 8.464 16 9 15.5 8.464 16 9.5 15.5 9.5 16 10 15.5 10.464 16 10.5 15.5 10.5 16 11 15.5 11 16 11.5 15.5 11.5 16 12 15.5 12 16 12.5 15.5 12.5 16 13 15.5 13 16 13.5 15.5 13.5 16 14 15.5 14 16 14.5 15.5 14.5 16 15 15.5 15 16 15.5 15.5 16 16 15.5 16 16 16.5 15.5 16.5 16 17 15.5 17 16 17.5 15.5 17.5 16 18 15.5 18 16 18.5 15.5 18.5 16 19 15.5 19 16 19.5 15.5 19.5 16 20 15.5 20 16 20.5 15.5 20.5 16 21 15.5 21 16 21.5 15.5 21.5 16 22 15.5 22 16 22.5 15.5 22.5 16 23 15.5 23 16 23.5 15.5 23.5 16 24 15.5 24 16 24.5 15.5 24.5 16 25 15.5 25 16 25.5 15.5 25.5 16 26 15.5 26 16 26.5 15.5 26.5 16 27 15.5 27 16 27.5 15.5 27.5 16 28 15.5 28 16 28.5 15.5 28.5 16 29 15.5 29 16 29.5 15.5 29.5 16 30 15.5 30 16 30.5 15.5 30.5 16 31 15.5 31 16 31.5 15.5 31.5 16 32 15.5 32 16 32.5 15.5 32.5 16 33 15.5 33 16 33.5 15.5 33.5 16 34 15.5 34 16 34.5 15.5 34.5 16 35 15.5 35 16 35.5 15.5 35.5 16 36 15.5 36 16 36.5 15.5 36.5 16 37 15.5 37 16 37.5 15.5 37.5 16 38 15.5 38 16 38.5 15.5 38.5 16 39 15.5 39 16 39.5 15.5 39.5 16 40 15.5 40 16 40.5 15.5 40.5 16 41 15.5 41 16 41.5 15.5 41.5 16 42 15.5 42 16 42.5 15.5 42.5 16 43 15.5 43 16 43.5 15.5 43.5 16 44 15.5 44 16 44.5 15.5 44.5 16 45 15.5 45 16 45.5 15.5 45.5 16 46 15.5 46 16 46.5 15.5 46.5 16 47 15.5 47 16 47.5 15.5 47.5 16 48 15.5 48 16 48.5 15.5 48.5 16 49 15.5 49 16 49.5 15.5 49.5 16 50 15.5 50 16 50.5 15.5 50.5 16 51 15.5 51 16 51.5 15.5 51.5 16 52 15.5 52 16 52.5 15.5 52.5 16 53 15.5 53 16 53.5 15.5 53.5 16 54 15.5 54 16 54.5 15.5 54.5 16 55 15.5 55 16 55.5 15.5 55.5 16 56 15.5 56 16 56.5 15.5 56.5 16 57 15.5 57 16 57.5 15.5 57.5 16 58 15.5 58 16 58.5 15.5 58.5 16 59 15.5 59 16 59.5 15.5 59.5 16 60 15.5 60 16 60.5 15.5 60.5 16 61 15.5 61 16 61.5 15.5 61.5 16 62 15.5 62 16 62.5 15.5 62.5 16 63 15.5 63 16 63.5 15.5 63.5 16 64 15.5 64 16 64.5 15.5 64.5 16 65 15.5 65 16 65.5 15.5 65.5 16 66 15.5 66 16 66.5 15.5 66.5 16 67 15.5 67 16 67.5 15.5 67.5 16 68 15.5 68 16 68.5 15.5 68.5 16 69 15.5 69 16 69.5 15.5 69.5 16 70 15.5 70 16 70.5 15.5 70.5 16 71 15.5 71 16 71.5 15.5 71.5 16 72 15.5 72 16 72.5 15.5 72.5 16 73 15.5 73 16 73.5 15.5 73.5 16 74 15.5 74 16 74.5 15.5 74.5 16 75 15.5 75 16 75.5 15.5 75.5 16 76 15.5 76 16 76.5 15.5 76.5 16 77 15.5 77 16 77.5 15.5 77.5 16 78 15.5 78 16 78.5 15.5 78.5 16 79 15.5 79 16 79.5 15.5 79.5 16 80 15.5 80 16 80.5 15.5 80.5 16 81 15.5 81 16 81.5 15.5 81.5 16 82 15.5 82 16 82.5 15.5 82.5 16 83 15.5 83 16 83.5 15.5 83.5 16 84 15.5 84 16 84.5 15.5 84.5 16 85 15.5 85 16 85.5 15.5 85.5 16 86 15.5 86 16 86.5 15.5 86.5 16 87 15.5 87 16 87.5 15.5 87.5 16 88 15.5 88 16 88.5 15.5 88.5 16 89 15.5 89 16 89.5 15.5 89.5 16 90 15.5 90 16 90.5 15.5 90.5 16 91 15.5 91 16 91.5 15.5 91.5 16 92 15.5 92 16 92.5 15.5 92.5 16 93 15.5 93 16 93.5 15.5 93.5 16 94 15.5 94 16 94.5 15.5 94.5 16 95 15.5 95 16 95.5 15.5 95.5 16 96 15.5 96 16 96.5 15.5 96.5 16 97 15.5 97 16 97.5 15.5 97.5 16 98 15.5 98 16 98.5 15.5 98.5 16 99 15.5 99 16 99.5 15.5 99.5 16 100 15.5 100 16 100.5 15.5 100.5 16 101 15.5 101 16 101.5 15.5 101.5 16 102 15.5 102 16 102.5 15.5 102.5 16 103 15.5 103 16 103.5 15.5 103.5 16 104 15.5 104 16 104.5 15.5 104.5 16 105 15.5 105 16 105.5 15.5 105.5 16 106 15.5 106 16 106.5 15.5 106.5 16 107 15.5 107 16 107.5 15.5 107.5 16 108 15.5 108 16 108.5 15.5 108.5 16 109 15.5 109 16 109.5 15.5 109.5 16 110 15.5 110 16 110.5 15.5 110.5 16 111 15.5 111 16 111.5 15.5 111.5 16 112 15.5 112 16 112.5 15.5 112.5 16 113 15.5 113 16 113.5 15.5 113.5 16 114 15.5 114 16 114.5 15.5 114.5 16 115 15.5 115 16 115.5 15.5 115.5 16 116 15.5 116 16 116.5 15.5 116.5 16 117 15.5 117 16 117.5 15.5 117.5 16 118 15.5 118 16 118.5 15.5 118.5 16 119 15.5 119 16 119.5 15.5 119.5 16 120 15.5 120 16 120.5 15.5 120.5 16 121 15.5 121 16 121.5 15.5 121.5 16 122 15.5 122 16 122.5 15.5 122.5 16 123 15.5 123 16 123.5 15.5 123.5 16 124 15.5 124 16 124.5 15.5 124.5 16 125 15.5 125 16 125.5 15.5 125.5 16 126 15.5 126 16 126.5 15.5 126.5 16 127 15.5 127 16 127.5 15.5 127.5 16 128 15.5 128 16 128.5 15.5 128.5 16 129 15.5 129 16 129.5 15.5 129.5 16 130 15.5 130 16 130.5 15.5 130.5 16 131 15.5 131 16 131.5 15.5 131.5 16 132 15.5 132 16 132.5 15.5 132.5 16 133 15.5 133 16 133.5 15.5 133.5 16 134 15.5 134 16 134.5 15.5 134.5 16 135 15.5 135 16 135.5 15.5 135.5 16 136 15.5 136 16 136.5 15.5 136.5 16 137 15.5 137 16 137.5 15.5 137.5 16 138 15.5 138 16 138.5 15.5 138.5 16 139 15.5 139 16 139.5 15.5 139.5 16 140 15.5 140 16 140.5 15.5 140.5 16 141 15.5 141 16 141.5 15.5 141.5 16 142 15.5 142 16 142.5 15.5 142.5 16 143 15.5 143 16 143.5 15.5 143.5 16 144 15.5 144 16 144.5 15.5 144.5 16 145 15.5 145 16 145.5 15.5 145.5 16 146 15.5 146 16 146.5 15.5 146.5 16 147 15.5 147 16 147.5 15.5 147.5 16 148 15.5 148 16 148.5 15.5 148.5 16 149 15.5 149 16 149.5 15.5 149.5 16 150 15.5 150 16 150.5 15.5 150.5 16 151 15.5 151 16 151.5 15.5 151.5 16 152 15.5 152 16 152.5 15.5 152.5 16 153 15.5 153 16 153.5 15.5 153.5 16 154 15.5 154 16 154.5 15.5 154.5 16 155 15.5 155 16 155.5 15.5 155.5 16 156 15.5 156 16 156.5 15.5 156.5 16 157 15.5 157 16 157.5 15.5 157.5 16 158 15.5 158 16 158.5 15.5 158.5 16 159 15.5 159 16 159.5 15.5 159.5 16 160 15.5 160 16 160.5 15.5 160.5 16 161 15.5 161 16 161.5 15.5 161.5 16 162 15.5 162 16 162.5 15.5 162.5 16 163 15.5 163 16 163.5 15.5 163.5 16 164 15.5 164 16 164.5 15.5 164.5 16 165 15.5 165 16 165.5 15.5 165.5 16 166 15.5 166 16 166.5 15.5 166.5 16 167 15.5 167 16 167.5 15.5 167.5 16 168 15.5 168 16 168.5 15.5 168.5 16 169 15.5 169 16 169.5 15.5 169.5 16 170 15.5 170 16 170.5 15.5 170.5 16 171 15.5 171 16 171.5 15.5 171.5 16 172 15.5 172 16 172.5 15.5 172.5 16 173 15.5 173 16 173.5 15.5 173.5 16 174 15.5 174 16 174.5 15.5 174.5 16 175 15.5 175 16 175.5 15.5 175.5 16 176 15.5 176 16 176.5 15.5 176.5 16 177 15.5 177 16 177.5 15.5 177.5 16 178 15.5 178 16 178.5 15.5 178.5 16 179 15.5 179 16 179.5 15.5 179.5 16 180 15.5 180 16 180.5 15.5 180.5 16 181 15.5 181 16 181.5 15.5 181.5 16 182 15.5 182 16 182.5 15.5 182.5 16 183 15.5 183 16 183.5 15.5 183.5 16 184 15.5 184 16 184.5 15.5 184.5 16 185 15.5 185 16 185.5 15.5 185.5 16 186 15.5 186 16 186.5 15.5 186.5 16 187 15.5 187 16 187.5 15.5 187.5 16 188 15.5 188 16 188.5 15.5 188.5 16 189 15.5 189 16 189.5 15.5 189.5 16 190 15.5 190 16 190.5 15.5 190.5 16 191 15.5 191 16 191.5 15.5 191.5 16 192 15.5 192 16 192.5 15.5 192.5 16 193 15.5 193 16 193.5 15.5 193.5 16 194 15.5 194 16 194.5 15.5 194.5 16 195 15.5 195 16 195.5 15.5 195.5 16 196 15.5 196 16 196.5 15.5 196.5 16 197 15.5 197 16 197.5 15.5 197.5 16 198 15.5 198 16 198.5 15.5 198.5 16 199 15.5 199 16 199.5 15.5 199.5 16 200 15.5 200 16 200.5 15.5 200.5 16 201 15.5 201 16 201.5 15.5 201.5 16 202 15.5 202 16 202.5 15.5 202.5 16 203 15.5 203 16 203.5 15.5 203.5 16 204 15.5 204 16 204.5 15.5 204.5 16 205 15.5 205 16 205.5 15.5 205.5 16 206 15.5 206 16 206.5 15.5 206.5 16 207 15.5 207 16 207.5 15.5 207.5 16 208 15.5 208 16 208.5 15.5 208.5 16 209 15.5 209 16 209.5 15.5 209.5 16 210 15.5 210 16 210.5 15.5 210.5 16 211 15.5 211 16 211.5 15.5 211.5 16 212 15.5 212 16 212.5 15.5 212.5 16 213 15.5 213 16 213.5 15.5 213.5 16 214 15.5 214 16 214.5 15.5 214.5 16 215 15.5 215 16 215.5 15.5 215.5 16 216 15.5 216 16 216.5 15.5 216.5 16 217 15.5 217 16 217.5 15.5 217.5 16 218 15.5 218 16 218.5 15.5 218.5 16 219 15.5 219 16 219.5 15.5 219.5 16 220 15.5 220 16 220.5 15.5 220.5 16 221 15.5 221 16 221.5 15.5 221.5 16 222 15.5 222 16 222.5 15.5 222.5 16 223 15.5 223 16 223.5 15.5 223.5 16 224 15.5 224 16 224.5 15.5 224.5 16 225 15.5 225 16 225.5 15.5 225.5 16 226 15.5 226 16 226.5 15.5 226.5 16 227 15.5 227 16 227.5 15.5 227.5 16 228 15.5 228 16 228.5 15.5 228.5 16 229 15.5 229 16 229.5 15.5 229.5 16 230 15.5 230 16 230.5 15.5 230.5 16 231 15.5 231 16 231.5 15.5 231.5 16 232 15.5 232 16 232.5 15.5 232.5 16 233 15.5 233 16 233.5 15.5 233.5 16 234 15.5 234 16 234.5 15.5 234.5 16 235 15.5 235 16 235.5 15.5 235.5 16 236 15.5 236 16 236.5 15.5 236.5 16 237 15.5 237 16 237.5 15.5 237.5 16 238 15.5 238 16 238.5 15.5 238.5 16 239 15.5 239 16 239.5 15.5 239.5 16 240 15.5 240 16 240.5 15.5 240.5 16 241 15.5 241 16 241.5 15.5 241.5 16 242 15.5 242 16 242.5 15.5 242.5 16 243 15.5 243 16 243.5 15.5 243.5 16 244 15.5 244 16 244.5 15.5 244.5 16 245 15.5 245 16 245.5 15.5 245.5 16 246 15.5 246 16 246.5 15.5 246.5 16 247 15.5 247 16 247.5 15.5 247.5 16 248 15.5 248 16 248.5 15.5 248.5 16 249 15.5 249 16 249.5 15.5 249.5 16 250 15.5 250 16 250.5 15.5 250.5 16 251 15.5 251 16 251.5 15.5 251.5 16 252 15.5 252 16 252.5 15.5 252.5 16 253 15.5 253 16 253.5 15.5 253.5 16 254 15.5 254 16 254.5 15.5 254.5 16 255 15.5 255 16 255.5 15.5 255.5 16 256 15.5 256 16 256.5 15.5 256.5 16 257 15.5 257 16 257.5 15.5 257.5 16 258 15.5 258 16 258.5 15.5 258.5 16 259 15.5 259 16 259.5 15.5 259.5 16 260 15.5 260 16 260.5 15.5 260.5 16 261 15.5 261 16 261.5 15.5 261.5 16 262 15.5 262 16 262.5 15.5 262.5 16 263 15.5 263 16 263.5 15.5 263.5 16 264 15.5 264 16 264.5 15.5 264.5 16 265 15.5 265 16 265.5 15.5 265.5 16 266 15.5 266 16 266.5 15.5 266.5 16 267 15.5 267 16 267.5 15.5 267.5 16 268 15.5 268 16 268.5 15.5 268.5 16 269 15.5 269 16 269.5 15.5 269.5 16 270 15.5 270 16 270.5 15.5 270.5 16 271 15.5 271 16 271.5 15.5 271.5 16 272 15.5 272 16 272.5 15.5 272.5 16 273 15.5 273 16 273.5 15.5 273.5 16 274 15.5 274 16 274.5 15.5 274.5 16 275 15.5 275 16 275.5 15.5 275.5 16 276 15.5 276 16 276.5 15.5 276.5 16 277 15.5 277 16 277.5 15.5 277.5 16 278 15.5 278 16 278.5 15.5 278.5 16 279 15.5 279 16 279.5 15.5 279.5 16 280 15.5 280 16 280.5 15.5 280.5 16 281 15.5 281 16 281.5 15.5 281.5 16 282 15.5 282 16 282.5 15.5 282.5 16 283 15.5 283 16 283.5 15.5 283.5 16 284 15.5 284 16 284.5 15.5 284.5 16 285 15.5 285 16 285.5 15.5 285.5 16 286 15.5 286 16 286.5 15.5 286.5 16 287 15.5 287 16 287.5 15.5 287.5 16 288 15.5 288 16 288.5 15.5 288.5 16 289 15.5 289 16 289.5 15.5 289.5 16 290 15.5 290 16 290.5 15.5 290.5 16 291 15.5 291 16 291.5 15.5 291.5 16 292 15.5 292 16 292.5 15.5 292.5 16 293 15.5 293 16 293.5 15.5 293.5 16 294 15.5 294 16 294.5 15.5 294.5 16 295 15.5 295 16 295.5 15.5 295.5 16 296 15.5 296 16 296.5 15.5 296.5 16 297 15.5 297 16 297.5 15.5 297.5 16 298 15.5 298 16 298.5 15.5 298.5 16 299 15.5 299 16 299.5 15.5 299.5 16 300 15.5 300 16 300.5 15.5 300.5 16 301 15.5 301 16 301.5 15.5 301.5 16 302 15.5 302 16 302.5 15.5 302.5 16 303 15.5 303 16 303.5 15.5 303.5 16 304 15.5 304 16 304.5 15.5 304.5 16 305 15.5 305 16 305.5 15.5 305.5 16 306 15.5 306 16 306.5 15.5 306.5 16 307 15.5 307 16 307.5 15.5 307.5 16 308 15.5 308 16 308.5 15.5 308.5 16 309 15.5 309 16 309.5 15.5 309.5 16 310 15.5 310 16 310.5 15.5 310.5 16 311 15.5 311 16 311.5 15.5 311.5 16 312 15.5 312 16 312.5 15.5 312.5 16 313 15.5 313 16 313.5 15.5 313.5 16 314 15.5 314 16 314.5 15.5 314.5 16 315 15.5 315 16 315.5 15.5 315.5 16 316 15.5 316 16 316.5 15.5 316.5 16 317 15.5 317 16 317.5 15.5 317.5 16 318 15.5 318 16 318.5 15.5 318.5 16 319 15.5 319 16 319.5 15.5 319.5 16 320 15.5 320 16 320.5 15.5 320.5 16 321 15.5 321 16 321.5 15.5 321.5 16 322 15.5 322 16 322.5 15.5 322.5 16 323 15.5 323 16 323.5 15.5 323.5 16 324 15.5 324 16 324.5 15.5 324.5 16 325 15.5 325 16 325.5 15.5 325.5 16 326 15.5 326 16 326.5 15.5 326.5 16 327 15.5 327 16 327.5 15.5 327.5 16 328 15.5 328 16 328.5 15.5 328.5 16 329 15.5 329 16 329.5 15.5 329.5 16 330 15.5 330 16 330.5 15.5 330.5 16 331 15.5 331 16 331.5 15.5 331.5 16 332 15.5 332 16 332.5 15.5 332.5 16 333 15.5 333 16 333.5 15.5 333.5 16 334 15.5 334 16 334.5 15.5 334.5 16 335 15.5 335 16 335.5 15.5 335.5 16 336 15.5 336 16 336.5 15.5 336.5 16 337 1
```

```
Number-System-Converter > converter.html > html.dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
72   function parseToDecimal(str, base) {
87     if (parts.length > 1) {
88       let fracStr = parts[1];
89       let divisor = base;
90       for (let i = 0; i < fracStr.length; i++) {
91         fracPart += parseInt(fracStr[i], base) / divisor;
92         divisor *= base;
93       }
94     }
95
96     let total = intPart + fracPart;
97     return isNegative ? -total : total;
98   }
99
100   // Formatter to standardize output format
101   function formatBase(num, base) {
102     if (isNaN(num)) return "NaN";
103     const isNeg = num < 0;
104     const absVal = Math.abs(num);
105     let str = absVal.toString(base).toUpperCase();
106     return (isNeg ? '-' : '') + str;
107   }
108
109   function createCaseHTML(id) {
110     return `
111     <div id="case-${id}" data-var="" class="p-6 border border-slate-200 dark:border-indigo-800 rounded-xl bg-slate-50 dark:bg-slate-800 relative shadow-sm dark:shadow-inner transition-colors duration-300">
112       <div class="flex justify-between items-center mb-4">
113         <label class="case-label font-bold text-xl text-indigo-800 dark:text-purple-300 bg-indigo-100 dark:bg-indigo-900/50 px-4 py-1 rounded-full border border-indigo-200 dark:border-indigo-700">Input</label>
114         <button onclick="removeCase('${id})" class="remove-btn p-2 text-red-500 dark:text-red-400 hover:text-red-700 dark:hover:text-red-300 hover:bg-red-100 dark:hover:bg-red-900/30 rounded-lg hidden transition-colors duration-300">
115           <svg class="w-5 h-5 fill-none stroke-current" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
116             <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M6 18L18 6" />
117           </svg>
118         </button>
119       </div>
120
121       <div class="grid grid-cols-1 md:grid-cols-2 gap-4 mb-4">
122         <div>
123           <label class="block text-sm font-semibold text-slate-600 dark:text-indigo-300 mb-1">Value</label>
124           <input type="text" id="val-${id}" class="w-full bg-white dark:bg-slate-900 border border-slate-300 dark:border-indigo-700 p-3 rounded-lg text-lg font-mono text-slate-900 dark:text-white focus:outline-none focus:ring-2 focus:ring-indigo-500 dark:focus:ring-indigo-400" />
125         </div>
126         <div>
127           <label class="block text-sm font-semibold text-slate-600 dark:text-indigo-300 mb-1">Base System</label>
128           <select id="base-${id}" class="w-full bg-white dark:bg-slate-900 border border-slate-300 dark:border-indigo-700 p-3 rounded-lg text-lg text-slate-900 dark:text-white focus:outline-none focus:ring-2 focus:ring-indigo-500 dark:focus:ring-indigo-400">
129             <option value="2">Binary (Base 2)</option>
130           </select>
131         </div>
132       </div>
133     </div>`;
134   }
135
136   function reindexLabels() {
137     const cases = document.querySelectorAll('[id^="case-"]');
138     cases.forEach((caseEl, index) => {
139       const varLetter = String.fromCharCode(65 + index); // 0 = A, 1 = B, etc.
140       caseEl.setAttribute('data-var', varLetter);
141       const labelEl = caseEl.querySelector('.case-label');
142       if (labelEl) {
143         labelEl.textContent = `Input ${varLetter}`;
144       }
145     });
146   }
147
148   function addCase() {
149     caseCounter++;
150     const container = document.getElementById('cases-container');
151     container.insertAdjacentHTML('beforeend', createCaseHTML(caseCounter));
152     reindexLabels();
153     updateRemoveButtons();
154     calculateTotal();
155   }
156
157   function removeCase(id) {
158     const caseEl = document.getElementById(`case-${id}`);
159     if (caseEl && document.querySelectorAll('[id^="case-"]').length > 2) {
160       caseEl.remove();
161       reindexLabels();
162     }
163   }
164
165   // Initialize
166   reindexLabels();
167   calculateTotal();
168   updateRemoveButtons();
169   addCase();
170
171   // Event listeners
172   document.getElementById('add-btn').addEventListener('click', addCase);
173   document.getElementById('remove-btn').addEventListener('click', removeCase);
174   document.getElementById('base-select').addEventListener('change', calculateTotal);
175   document.getElementById('val-input').addEventListener('input', calculateTotal);
176
177   // Calculate total
178   function calculateTotal() {
179     let total = 0;
180     const cases = document.querySelectorAll('[id^="case-"]');
181     cases.forEach((caseEl) => {
182       const val = document.getElementById(`val-${caseEl.id.split('-')[1]}`).value;
183       const base = document.getElementById(`base-${caseEl.id.split('-')[1]}`).value;
184       const varLetter = caseEl.getAttribute('data-var');
185       if (val && base) {
186         total += parseInt(val, base) * Math.pow(base, cases.length - caseEl.id.split('-')[1]);
187       }
188     });
189     document.getElementById('total').value = formatBase(total, 10);
190   }
191
192   // Update remove buttons
193   function updateRemoveButtons() {
194     const cases = document.querySelectorAll('[id^="case-"]');
195     cases.forEach((caseEl) => {
196       const removeBtn = document.getElementById(`remove-${caseEl.id.split('-')[1]}`);
197       if (removeBtn) {
198         removeBtn.classList.toggle('hidden', cases.length < 2);
199       }
200     });
201   }
202
203   // Initial calculation
204   calculateTotal();
205   updateRemoveButtons();
206
207   // Close modal
208   document.getElementById('close-modal').addEventListener('click', () => {
209     document.getElementById('modal').classList.add('hidden');
210   });
211
212   // Prevent closing on click
213   document.getElementById('modal').addEventListener('click', (e) => {
214     e.stopPropagation();
215   });
216
217   // Prevent closing on keypress
218   document.addEventListener('keydown', (e) => {
219     if (e.key === 'Escape') {
220       document.getElementById('modal').classList.add('hidden');
221     }
222   });
223
224   // Prevent closing on scroll
225   document.getElementById('modal').addEventListener('wheel', (e) => {
226     e.stopPropagation();
227   });
228
229   // Prevent closing on touch
230   document.getElementById('modal').addEventListener('touchmove', (e) => {
231     e.stopPropagation();
232   });
233
234   // Prevent closing on drag
235   document.getElementById('modal').addEventListener('drag', (e) => {
236     e.stopPropagation();
237   });
238
239   // Prevent closing on drop
240   document.getElementById('modal').addEventListener('drop', (e) => {
241     e.stopPropagation();
242   });
243
244   // Prevent closing on paste
245   document.getElementById('modal').addEventListener('paste', (e) => {
246     e.stopPropagation();
247   });
248
249   // Prevent closing on copy
250   document.getElementById('modal').addEventListener('copy', (e) => {
251     e.stopPropagation();
252   });
253
254   // Prevent closing on cut
255   document.getElementById('modal').addEventListener('cut', (e) => {
256     e.stopPropagation();
257   });
258
259   // Prevent closing on paste
260   document.getElementById('modal').addEventListener('paste', (e) => {
261     e.stopPropagation();
262   });
263
264   // Prevent closing on copy
265   document.getElementById('modal').addEventListener('copy', (e) => {
266     e.stopPropagation();
267   });
268
269   // Prevent closing on cut
270   document.getElementById('modal').addEventListener('cut', (e) => {
271     e.stopPropagation();
272   });
273
274   // Prevent closing on paste
275   document.getElementById('modal').addEventListener('paste', (e) => {
276     e.stopPropagation();
277   });
278
279   // Prevent closing on copy
280   document.getElementById('modal').addEventListener('copy', (e) => {
281     e.stopPropagation();
282   });
283
284   // Prevent closing on cut
285   document.getElementById('modal').addEventListener('cut', (e) => {
286     e.stopPropagation();
287   });
288
289   // Prevent closing on paste
290   document.getElementById('modal').addEventListener('paste', (e) => {
291     e.stopPropagation();
292   });
293
294   // Prevent closing on copy
295   document.getElementById('modal').addEventListener('copy', (e) => {
296     e.stopPropagation();
297   });
298
299   // Prevent closing on cut
300   document.getElementById('modal').addEventListener('cut', (e) => {
301     e.stopPropagation();
302   });
303
304   // Prevent closing on paste
305   document.getElementById('modal').addEventListener('paste', (e) => {
306     e.stopPropagation();
307   });
308
309   // Prevent closing on copy
310   document.getElementById('modal').addEventListener('copy', (e) => {
311     e.stopPropagation();
312   });
313
314   // Prevent closing on cut
315   document.getElementById('modal').addEventListener('cut', (e) => {
316     e.stopPropagation();
317   });
318
319   // Prevent closing on paste
320   document.getElementById('modal').addEventListener('paste', (e) => {
321     e.stopPropagation();
322   });
323
324   // Prevent closing on copy
325   document.getElementById('modal').addEventListener('copy', (e) => {
326     e.stopPropagation();
327   });
328
329   // Prevent closing on cut
330   document.getElementById('modal').addEventListener('cut', (e) => {
331     e.stopPropagation();
332   });
333
334   // Prevent closing on paste
335   document.getElementById('modal').addEventListener('paste', (e) => {
336     e.stopPropagation();
337   });
338
339   // Prevent closing on copy
340   document.getElementById('modal').addEventListener('copy', (e) => {
341     e.stopPropagation();
342   });
343
344   // Prevent closing on cut
345   document.getElementById('modal').addEventListener('cut', (e) => {
346     e.stopPropagation();
347   });
348
349   // Prevent closing on paste
350   document.getElementById('modal').addEventListener('paste', (e) => {
351     e.stopPropagation();
352   });
353
354   // Prevent closing on copy
355   document.getElementById('modal').addEventListener('copy', (e) => {
356     e.stopPropagation();
357   });
358
359   // Prevent closing on cut
360   document.getElementById('modal').addEventListener('cut', (e) => {
361     e.stopPropagation();
362   });
363
364   // Prevent closing on paste
365   document.getElementById('modal').addEventListener('paste', (e) => {
366     e.stopPropagation();
367   });
368
369   // Prevent closing on copy
370   document.getElementById('modal').addEventListener('copy', (e) => {
371     e.stopPropagation();
372   });
373
374   // Prevent closing on cut
375   document.getElementById('modal').addEventListener('cut', (e) => {
376     e.stopPropagation();
377   });
378
379   // Prevent closing on paste
380   document.getElementById('modal').addEventListener('paste', (e) => {
381     e.stopPropagation();
382   });
383
384   // Prevent closing on copy
385   document.getElementById('modal').addEventListener('copy', (e) => {
386     e.stopPropagation();
387   });
388
389   // Prevent closing on cut
390   document.getElementById('modal').addEventListener('cut', (e) => {
391     e.stopPropagation();
392   });
393
394   // Prevent closing on paste
395   document.getElementById('modal').addEventListener('paste', (e) => {
396     e.stopPropagation();
397   });
398
399   // Prevent closing on copy
400   document.getElementById('modal').addEventListener('copy', (e) => {
401     e.stopPropagation();
402   });
403
404   // Prevent closing on cut
405   document.getElementById('modal').addEventListener('cut', (e) => {
406     e.stopPropagation();
407   });
408
409   // Prevent closing on paste
410   document.getElementById('modal').addEventListener('paste', (e) => {
411     e.stopPropagation();
412   });
413
414   // Prevent closing on copy
415   document.getElementById('modal').addEventListener('copy', (e) => {
416     e.stopPropagation();
417   });
418
419   // Prevent closing on cut
420   document.getElementById('modal').addEventListener('cut', (e) => {
421     e.stopPropagation();
422   });
423
424   // Prevent closing on paste
425   document.getElementById('modal').addEventListener('paste', (e) => {
426     e.stopPropagation();
427   });
428
429   // Prevent closing on copy
430   document.getElementById('modal').addEventListener('copy', (e) => {
431     e.stopPropagation();
432   });
433
434   // Prevent closing on cut
435   document.getElementById('modal').addEventListener('cut', (e) => {
436     e.stopPropagation();
437   });
438
439   // Prevent closing on paste
440   document.getElementById('modal').addEventListener('paste', (e) => {
441     e.stopPropagation();
442   });
443
444   // Prevent closing on copy
445   document.getElementById('modal').addEventListener('copy', (e) => {
446     e.stopPropagation();
447   });
448
449   // Prevent closing on cut
450   document.getElementById('modal').addEventListener('cut', (e) => {
451     e.stopPropagation();
452   });
453
454   // Prevent closing on paste
455   document.getElementById('modal').addEventListener('paste', (e) => {
456     e.stopPropagation();
457   });
458
459   // Prevent closing on copy
460   document.getElementById('modal').addEventListener('copy', (e) => {
461     e.stopPropagation();
462   });
463
464   // Prevent closing on cut
465   document.getElementById('modal').addEventListener('cut', (e) => {
466     e.stopPropagation();
467   });
468
469   // Prevent closing on paste
470   document.getElementById('modal').addEventListener('paste', (e) => {
471     e.stopPropagation();
472   });
473
474   // Prevent closing on copy
475   document.getElementById('modal').addEventListener('copy', (e) => {
476     e.stopPropagation();
477   });
478
479   // Prevent closing on cut
480   document.getElementById('modal').addEventListener('cut', (e) => {
481     e.stopPropagation();
482   });
483
484   // Prevent closing on paste
485   document.getElementById('modal').addEventListener('paste', (e) => {
486     e.stopPropagation();
487   });
488
489   // Prevent closing on copy
490   document.getElementById('modal').addEventListener('copy', (e) => {
491     e.stopPropagation();
492   });
493
494   // Prevent closing on cut
495   document.getElementById('modal').addEventListener('cut', (e) => {
496     e.stopPropagation();
497   });
498
499   // Prevent closing on paste
500   document.getElementById('modal').addEventListener('paste', (e) => {
501     e.stopPropagation();
502   });
503
504   // Prevent closing on copy
505   document.getElementById('modal').addEventListener('copy', (e) => {
506     e.stopPropagation();
507   });
508
509   // Prevent closing on cut
510   document.getElementById('modal').addEventListener('cut', (e) => {
511     e.stopPropagation();
512   });
513
514   // Prevent closing on paste
515   document.getElementById('modal').addEventListener('paste', (e) => {
516     e.stopPropagation();
517   });
518
519   // Prevent closing on copy
520   document.getElementById('modal').addEventListener('copy', (e) => {
521     e.stopPropagation();
522   });
523
524   // Prevent closing on cut
525   document.getElementById('modal').addEventListener('cut', (e) => {
526     e.stopPropagation();
527   });
528
529   // Prevent closing on paste
530   document.getElementById('modal').addEventListener('paste', (e) => {
531     e.stopPropagation();
532   });
533
534   // Prevent closing on copy
535   document.getElementById('modal').addEventListener('copy', (e) => {
536     e.stopPropagation();
537   });
538
539   // Prevent closing on cut
540   document.getElementById('modal').addEventListener('cut', (e) => {
541     e.stopPropagation();
542   });
543
544   // Prevent closing on paste
545   document.getElementById('modal').addEventListener('paste', (e) => {
546     e.stopPropagation();
547   });
548
549   // Prevent closing on copy
550   document.getElementById('modal').addEventListener('copy', (e) => {
551     e.stopPropagation();
552   });
553
554   // Prevent closing on cut
555   document.getElementById('modal').addEventListener('cut', (e) => {
556     e.stopPropagation();
557   });
558
559   // Prevent closing on paste
560   document.getElementById('modal').addEventListener('paste', (e) => {
561     e.stopPropagation();
562   });
563
564   // Prevent closing on copy
565   document.getElementById('modal').addEventListener('copy', (e) => {
566     e.stopPropagation();
567   });
568
569   // Prevent closing on cut
570   document.getElementById('modal').addEventListener('cut', (e) => {
571     e.stopPropagation();
572   });
573
574   // Prevent closing on paste
575   document.getElementById('modal').addEventListener('paste', (e) => {
576     e.stopPropagation();
577   });
578
579   // Prevent closing on copy
580   document.getElementById('modal').addEventListener('copy', (e) => {
581     e.stopPropagation();
582   });
583
584   // Prevent closing on cut
585   document.getElementById('modal').addEventListener('cut', (e) => {
586     e.stopPropagation();
587   });
588
589   // Prevent closing on paste
590   document.getElementById('modal').addEventListener('paste', (e) => {
591     e.stopPropagation();
592   });
593
594   // Prevent closing on copy
595   document.getElementById('modal').addEventListener('copy', (e) => {
596     e.stopPropagation();
597   });
598
599   // Prevent closing on cut
600   document.getElementById('modal').addEventListener('cut', (e) => {
601     e.stopPropagation();
602   });
603
604   // Prevent closing on paste
605   document.getElementById('modal').addEventListener('paste', (e) => {
606     e.stopPropagation();
607   });
608
609   // Prevent closing on copy
610   document.getElementById('modal').addEventListener('copy', (e) => {
611     e.stopPropagation();
612   });
613
614   // Prevent closing on cut
615   document.getElementById('modal').addEventListener('cut', (e) => {
616     e.stopPropagation();
617   });
618
619   // Prevent closing on paste
620   document.getElementById('modal').addEventListener('paste', (e) => {
621     e.stopPropagation();
622   });
623
624   // Prevent closing on copy
625   document.getElementById('modal').addEventListener('copy', (e) => {
626     e.stopPropagation();
627   });
628
629   // Prevent closing on cut
630   document.getElementById('modal').addEventListener('cut', (e) => {
631     e.stopPropagation();
632   });
633
634   // Prevent closing on paste
635   document.getElementById('modal').addEventListener('paste', (e) => {
636     e.stopPropagation();
637   });
638
639   // Prevent closing on copy
640   document.getElementById('modal').addEventListener('copy', (e) => {
641     e.stopPropagation();
642   });
643
644   // Prevent closing on cut
645   document.getElementById('modal').addEventListener('cut', (e) => {
646     e.stopPropagation();
647   });
648
649   // Prevent closing on paste
650   document.getElementById('modal').addEventListener('paste', (e) => {
651     e.stopPropagation();
652   });
653
654   // Prevent closing on copy
655   document.getElementById('modal').addEventListener('copy', (e) => {
656     e.stopPropagation();
657   });
658
659   // Prevent closing on cut
660   document.getElementById('modal').addEventListener('cut', (e) => {
661     e.stopPropagation();
662   });
663
664   // Prevent closing on paste
665   document.getElementById('modal').addEventListener('paste', (e) => {
666     e.stopPropagation();
667   });
668
669   // Prevent closing on copy
670   document.getElementById('modal').addEventListener('copy', (e) => {
671     e.stopPropagation();
672   });
673
674   // Prevent closing on cut
675   document.getElementById('modal').addEventListener('cut', (e) => {
676     e.stopPropagation();
677   });
678
679   // Prevent closing on paste
680   document.getElementById('modal').addEventListener('paste', (e) => {
681     e.stopPropagation();
682   });
683
684   // Prevent closing on copy
685   document.getElementById('modal').addEventListener('copy', (e) => {
686     e.stopPropagation();
687   });
688
689   // Prevent closing on cut
690   document.getElementById('modal').addEventListener('cut', (e) => {
691     e.stopPropagation();
692   });
693
694   // Prevent closing on paste
695   document.getElementById('modal').addEventListener('paste', (e) => {
696     e.stopPropagation();
697   });
698
699   // Prevent closing on copy
700   document.getElementById('modal').addEventListener('copy', (e) => {
701     e.stopPropagation();
702   });
703
704   // Prevent closing on cut
705   document.getElementById('modal').addEventListener('cut', (e) => {
706     e.stopPropagation();
707   });
708
709   // Prevent closing on paste
710   document.getElementById('modal').addEventListener('paste', (e) => {
711     e.stopPropagation();
712   });
713
714   // Prevent closing on copy
715   document.getElementById('modal').addEventListener('copy', (e) => {
716     e.stopPropagation();
717   });
718
719   // Prevent closing on cut
720   document.getElementById('modal').addEventListener('cut', (e) => {
721     e.stopPropagation();
722   });
723
724   // Prevent closing on paste
725   document.getElementById('modal').addEventListener('paste', (e) => {
726     e.stopPropagation();
727   });
728
729   // Prevent closing on copy
730   document.getElementById('modal').addEventListener('copy', (e) => {
731     e.stopPropagation();
732   });
733
734   // Prevent closing on cut
735   document.getElementById('modal').addEventListener('cut', (e) => {
736     e.stopPropagation();
737   });
738
739   // Prevent closing on paste
740   document.getElementById('modal').addEventListener('paste', (e) => {
741     e.stopPropagation();
742   });
743
744   // Prevent closing on copy
745   document.getElementById('modal').addEventListener('copy', (e) => {
746     e.stopPropagation();
747   });
748
749   // Prevent closing on cut
750   document.getElementById('modal').addEventListener('cut', (e) => {
751     e.stopPropagation();
752   });
753
754   // Prevent closing on paste
755   document.getElementById('modal').addEventListener('paste', (e) => {
756     e.stopPropagation();
757   });
758
759   // Prevent closing on copy
760   document.getElementById('modal').addEventListener('copy', (e) => {
761     e.stopPropagation();
762   });
763
764   // Prevent closing on cut
765   document.getElementById('modal').addEventListener('cut', (e) => {
766     e.stopPropagation();
767   });
768
769   // Prevent closing on paste
770   document.getElementById('modal').addEventListener('paste', (e) => {
771     e.stopPropagation();
772   });
773
774   // Prevent closing on copy
775   document.getElementById('modal').addEventListener('copy', (e) => {
776     e.stopPropagation();
777   });
778
779   // Prevent closing on cut
780   document.getElementById('modal').addEventListener('cut', (e) => {
781     e.stopPropagation();
782   });
783
784   // Prevent closing on paste
785   document.getElementById('modal').addEventListener('paste', (e) => {
786     e.stopPropagation();
787   });
788
789   // Prevent closing on copy
790   document.getElementById('modal').addEventListener('copy', (e) => {
791     e.stopPropagation();
792   });
793
794   // Prevent closing on cut
795   document.getElementById('modal').addEventListener('cut', (e) => {
796     e.stopPropagation();
797   });
798
799   // Prevent closing on paste
800   document.getElementById('modal').addEventListener('paste', (e) => {
801     e.stopPropagation();
802   });
803
804   // Prevent closing on copy
805   document.getElementById('modal').addEventListener('copy', (e) => {
806     e.stopPropagation();
807   });
808
809   // Prevent closing on cut
810   document.getElementById('modal').addEventListener('cut', (e) => {
811     e.stopPropagation();
812   });
813
814   // Prevent closing on paste
815   document.getElementById('modal').addEventListener('paste', (e) => {
816     e.stopPropagation();
817   });
818
819   // Prevent closing on copy
820   document.getElementById('modal').addEventListener('copy', (e) => {
821     e.stopPropagation();
822   });
823
824   // Prevent closing on cut
825   document.getElementById('modal').addEventListener('cut', (e) => {
826     e.stopPropagation();
827   });
828
829   // Prevent closing on paste
830   document.getElementById('modal').addEventListener('paste', (e) => {
831     e.stopPropagation();
832   });
833
834   // Prevent closing on copy
835   document.getElementById('modal').addEventListener('copy', (e) => {
836     e.stopPropagation();
837   });
838
839   // Prevent closing on cut
840   document.getElementById('modal').addEventListener('cut', (e) => {
841     e.stopPropagation();
842   });
843
844   // Prevent closing on paste
845   document.getElementById('modal').addEventListener('paste', (e) => {
846     e.stopPropagation();
847   });
848
849   // Prevent closing on copy
850   document.getElementById('modal').addEventListener('copy', (e) => {
851     e.stopPropagation();
852   });
853
854   // Prevent closing on cut
855   document.getElementById('modal').addEventListener('cut', (e) => {
856     e.stopPropagation();
857   });
858
859   // Prevent closing on paste
860   document.getElementById('modal').addEventListener('paste', (e) => {
861     e.stopPropagation();
862   });
863
864   // Prevent closing on copy
865   document.getElementById('modal').addEventListener('copy', (e) => {
866     e.stopPropagation();
867   });
868
869   // Prevent closing on cut
870   document.getElementById('modal').addEventListener('cut', (e) => {
871     e.stopPropagation();
872   });
873
874   // Prevent closing on paste
875   document.getElementById('modal').addEventListener('paste', (e) => {
876     e.stopPropagation();
877   });
878
879   // Prevent closing on copy
880   document.getElementById('modal').addEventListener('copy', (e) => {
881     e.stopPropagation();
882   });
883
884   // Prevent closing on cut
885   document.getElementById('modal').addEventListener('cut', (e) => {
886     e.stopPropagation();
887   });
888
889   // Prevent closing on paste
890   document.getElementById('modal').addEventListener('paste', (e) => {
891     e.stopPropagation();
892   });
893
894   // Prevent closing on copy
895   document.getElementById('modal').addEventListener('copy', (e) => {
896     e.stopPropagation();
897   });
898
899   // Prevent closing on cut
900   document.getElementById('modal').addEventListener('cut', (e) => {
901     e.stopPropagation();
902   });
903
904   // Prevent closing on paste
905   document.getElementById('modal').addEventListener('paste', (e) => {
906     e.stopPropagation();
907   });
908
909   // Prevent closing on copy
910   document.getElementById('modal').addEventListener('copy', (e) => {
911     e.stopPropagation();
912   });
913
914   // Prevent closing on cut
915   document.getElementById('modal').addEventListener('cut', (e) => {
916     e.stopPropagation();
917   });
918
919   // Prevent closing on paste
920   document.getElementById('modal').addEventListener('paste', (e) => {
921     e.stopPropagation();
922   });
923
924   // Prevent closing on copy
925   document.getElementById('modal').addEventListener('copy', (e) => {
926     e.stopPropagation();
927   });
928
929   // Prevent closing on cut
930   document.getElementById('modal').addEventListener('cut', (e) => {
931     e.stopPropagation();
932   });
933
934   // Prevent closing on paste
935   document.getElementById('modal').addEventListener('paste', (e) => {
936     e.stopPropagation();
937   });
938
939   // Prevent closing on copy
940   document.getElementById('modal').addEventListener('copy', (e) => {
941     e.stopPropagation();
942   });
943
944   // Prevent closing on cut
945   document.getElementById('modal').addEventListener('cut', (e) => {
946     e.stopPropagation();
947   });
948
949   // Prevent closing on paste
950   document.getElementById('modal').addEventListener('paste', (e) => {
951     e.stopPropagation();
952   });
953
954   // Prevent closing on copy
955   document.getElementById('modal').addEventListener('copy', (e) => {
956     e.stopPropagation();
957   });
958
959   // Prevent closing on cut
960   document.getElementById('modal').addEventListener('cut', (e) => {
961     e.stopPropagation();
962   });
963
964   // Prevent closing on paste
965   document.getElementById('modal').addEventListener('paste', (e) => {
966     e.stopPropagation();
967   });
968
969   // Prevent closing on copy
970   document.getElementById('modal').addEventListener('copy', (e) => {
971     e.stopPropagation();
972   });
973
974   // Prevent closing on cut
975   document.getElementById('modal').addEventListener('cut', (e) => {
976     e.stopPropagation();
977   });
978
979   // Prevent closing on paste
980   document.getElementById('modal').addEventListener('paste', (e) => {
981     e.stopPropagation();
982   });
983
984   // Prevent closing on copy
985   document.getElementById('modal').addEventListener('copy', (e) => {
986     e.stopPropagation();
987   });
988
989   // Prevent closing on cut
990   document.getElementById('modal').addEventListener('cut', (e) => {
991     e.stopPropagation();
992   });
993
994   // Prevent closing on paste
995   document.getElementById('modal').addEventListener('paste', (e) => {
996     e.stopPropagation();
997   });
998
999   // Prevent closing on copy
1000   document.getElementById('modal').addEventListener('copy', (e) => {
1001     e.stopPropagation();
1002   });
1003
1004   // Prevent closing on cut
1005   document.getElementById('modal').addEventListener('cut', (e) => {
1006     e.stopPropagation();
1007   });
1008
1009   // Prevent closing on paste
1010   document.getElementById('modal').addEventListener('paste', (e) => {
1011     e.stopPropagation();
1012   });
1013
1014   // Prevent closing on copy
1015   document.getElementById('modal').addEventListener('copy', (e) => {
1016     e.stopPropagation();
1017   });
1018
1019   // Prevent closing on cut
1020   document.getElementById('modal').addEventListener('cut', (e) => {
1021     e.stopPropagation();
1022   });
1023
1024   // Prevent closing on paste
1025   document.getElementById('modal').addEventListener('paste', (e) => {
1026     e.stopPropagation();
1027   });
1028
1029   // Prevent closing on copy
1030   document.getElementById('modal').addEventListener('copy', (e) => {
1031     e.stopPropagation();
1032   });
1033
1034   // Prevent closing on cut
1035   document.getElementById('modal').addEventListener('cut', (e) => {
1036     e.stopPropagation();
1037   });
1038
1039   // Prevent closing on paste
1040   document.getElementById('modal').addEventListener('paste', (e) => {
1041     e.stopPropagation();
1042   });
1043
1044   // Prevent closing on copy
1045   document.getElementById('modal').addEventListener('copy', (e) => {
1046     e.stopPropagation();
1047   });
1048
1049   // Prevent closing on cut
1050   document.getElementById('modal').addEventListener('cut', (e) => {
1051     e.stopPropagation();
1052   });
1053
1054   // Prevent closing on paste
1055   document.getElementById('modal').addEventListener('paste', (e) => {
1056     e.stopPropagation();
1057   });
1058
1059   // Prevent closing on copy
1060   document.getElementById('modal').addEventListener('copy', (e) => {
1061     e.stopPropagation();
1062   });
1063
1064   // Prevent closing on cut
1065   document.getElementById('modal').addEventListener('cut', (e) => {
1066     e.stopPropagation();
1067   });
1068
1069   // Prevent closing on paste
1070   document.getElementById('modal').addEventListener('paste', (e) => {
1071     e.stopPropagation();
1072   });
1073
1074   // Prevent closing on copy
1075   document.getElementById('modal').addEventListener('copy', (e) => {
1076     e.stopPropagation();
1077   });
1078
1079   // Prevent closing on cut
1080   document.getElementById('modal').addEventListener('cut', (e) => {
1081     e.stopPropagation();
1082   });
1083
1084   // Prevent closing on paste
1085   document.getElementById('modal').addEventListener('paste', (e) => {
1086     e.stopPropagation();
1087   });
1088
1089   // Prevent closing on copy
1090   document.getElementById('modal').addEventListener('copy', (e) => {
1091     e.stopPropagation();
1092   });
1093
1094   // Prevent closing on cut
1095   document.getElementById('modal').addEventListener('cut', (e) => {
1096     e.stopPropagation();
1097   });
1098
1099   // Prevent closing on paste
1100   document.getElementById('modal').addEventListener('paste', (e) => {
1101     e.stopPropagation();
1102   });
1103
1104   // Prevent closing on copy
1105   document.getElementById('modal').addEventListener('copy', (e) => {
1106     e.stopPropagation();
1107   });
1108
1109   // Prevent closing on cut
1110   document.getElementById('modal').addEventListener('cut', (e) => {
1111     e.stopPropagation();
1112   });
1113
1114   // Prevent closing on paste
1115   document.getElementById('modal').addEventListener('paste', (e) => {
1116     e.stopPropagation();
1117   });
1118
1119   // Prevent closing on copy
1120   document.getElementById('modal').addEventListener('copy', (e) => {
1121     e.stopPropagation();
1122   });
1123
1124   // Prevent closing on cut
1125   document.getElementById('modal').addEventListener('cut', (e) => {
1126     e.stopPropagation();
1127   });
1128
1129   // Prevent closing on paste
1130   document.getElementById('modal').addEventListener('paste', (e) => {
1131     e.stopPropagation();
1132   });
1133
1134   // Prevent closing on copy
1135   document.getElementById('modal').addEventListener('copy', (e) => {
1136     e.stopPropagation();
1137   });
1138
1139   // Prevent closing on cut
1140   document.getElementById('modal').addEventListener('cut', (e) => {
1141     e.stopPropagation();
1142   });
1143
1144   // Prevent closing on paste
1145   document.getElementById('modal').addEventListener('paste', (e) => {
1146     e.stopPropagation();
1147   });
1148
1149   // Prevent closing on copy
1150   document.getElementById('modal').addEventListener('copy', (e) => {
1151     e.stopPropagation();
1152   });
1153
1154   // Prevent closing on cut
1155   document.getElementById('modal').addEventListener('cut', (e) => {
1156     e.stopPropagation();
1157   });
1158
1159   // Prevent closing on paste
1160   document.getElementById('modal').addEventListener('paste', (e) => {
1161     e.stopPropagation();
1162   });
1163
1164   // Prevent closing on copy
1165   document.getElementById('modal').addEventListener('copy', (e) => {
1166     e.stopPropagation();
1167   });
1168
1169   // Prevent closing on cut
1170   document.getElementById('modal').addEventListener('cut', (e) => {
1171     e.stopPropagation();
1172   });
1173
1174   // Prevent closing on paste
1175   document.getElementById('modal').addEventListener('paste', (e) => {
1176     e.stopPropagation();
1177   });
1178
1179   // Prevent closing on copy
1180   document.getElementById('modal').addEventListener('copy', (e) => {
1181     e.stopPropagation();
1182   });
1183
1184   // Prevent closing on cut
1185   document.getElementById('modal').addEventListener('cut', (e) => {
1186     e.stopPropagation();
1187   });
1188
1189   // Prevent closing on paste
1190   document.getElementById('modal').addEventListener('paste', (e) => {
1191     e.stopPropagation();
1192   });
1193
1194   // Prevent closing on copy
1195   document.getElementById('modal').addEventListener('copy', (e) => {
1196     e.stopPropagation();
1197   });
1198
1199   // Prevent closing on cut
1200   document.getElementById('modal').addEventListener('cut', (e) => {
1201     e.stopPropagation();
1202   });
1203
1204   // Prevent closing on paste
1205   document.getElementById('modal').addEventListener('paste', (e) => {
1206     e.stopPropagation();
1207   });
1208
1209   // Prevent closing on copy
1210   document.getElementById('modal').addEventListener('copy', (e) => {
1211     e.stopPropagation();
1212   });
1213
1214   // Prevent closing on cut
1215   document.getElementById('modal').addEventListener('cut', (e) => {
1216     e.stopPropagation();
1217   });
1218
1219   // Prevent closing on paste
1220   document.getElementById('modal').addEventListener('paste', (e) => {
1221     e.stopPropagation();
1222   });
1223
1224   // Prevent closing on copy
1225   document.getElementById('modal').addEventListener('copy', (e) => {
122
```

```
File Edit Selection View Go Run Terminal Help
Number-System-Converter > converter.html > html:dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
165 function removeUseless() {
173 }
174
175 function updateRemoveButtons() {
176   const buttons = document.querySelectorAll('.remove-btn');
177   if (buttons.length <= 2) {
178     buttons.forEach(btn => btn.classList.add('hidden'));
179   } else {
180     buttons.forEach(btn => btn.classList.remove('hidden'));
181   }
182 }
183
184 function handleInput(id) {
185   convertIndividualNumber(id);
186   calculateTotal();
187 }
188
189 function convertIndividualNumber(id) {
190   const rawValue = document.getElementById('val-' + id).value.trim();
191   const inBase = parseInt(document.getElementById('inBase-' + id).value);
192   const outputDiv = document.getElementById('out-' + id);
193
194   if (rawValue === "") {
195     outputDiv.innerHTML = "Awaiting input...";
196     outputDiv.className = "mt-4 p-4 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] flex items-center justify-center text-slate-500 dark:text-indigo-400 text-lg";
197     return;
198   }
199
200   // Regex updated to support optional fractional parts (decimals)
201   let isValid = false;
202   if (inBase === 2) isValid = /^[01]*(\.[01]+)?$/ .test(rawValue);
203   if (inBase === 8) isValid = /^[0-7]*(\.[0-7]+)?$/ .test(rawValue);
204   if (inBase === 10) isValid = /^[0-9]*(\.[0-9]+)?$/ .test(rawValue);
205   if (inBase === 16) isValid = /^[0-9a-fA-F]*(\.[0-9a-fA-F]+)?$/ .test(rawValue);
206
207   if (!isValid) {
208     outputDiv.innerHTML = "<span class='text-red-600 dark:text-red-400 font-bold'>Invalid input for selected base.</span>";
209     outputDiv.className = "mt-4 p-4 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] flex items-center justify-center transition-colors";
210     return;
211   }
212
213   let decValue = parseToDecimal(rawValue, inBase);
214
215   const binStr = formatBase(decValue, 2);
216   const octStr = formatBase(decValue, 8);
217   const decStr = formatBase(decValue, 10);
218   const hexStr = formatBase(decValue, 16);
219
220   outputDiv.innerHTML = `
221     <div class="grid grid-cols-2 lg:grid-cols-4 gap-3 w-full">
222       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
223         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Binary</span>
224         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${binStr}</span></div>
225       </div>
226       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
227         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Octal</span>
228         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${octStr}</span></div>
229       </div>
230       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
231         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Decimal</span>
232         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${decStr}</span></div>
233       </div>
234       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
235         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Hexadecimal</span>
236         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${hexStr}</span></div>
237       </div>
238     </div>
239   `;
240   outputDiv.className = "mt-4 p-3 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] transition-colors";
241 }
242
243 function evaluatePostfix(postfix, variables) {
244   const valStack = [];
245   for (let token of postfix) {
246     if (typeof token === 'number') {
247       valStack.push(token);
248     } else if (variables.hasOwnProperty(token)) {
249       valStack.push(variables[token]);
250     } else {
251       if (valStack.length < 2) throw new Error("Invalid expression syntax.");
252       const b = valStack.pop();
253       const a = valStack.pop();
254       if (token === '+') valStack.push(a + b);
255       else if (token === '-') valStack.push(a - b);
256       else if (token === '*') valStack.push(a * b);
257       else if (token === '/') {
258         if (b === 0) throw new Error("Division by zero error.");
259       }
260     }
261   }
262   return valStack[0];
263 }
```

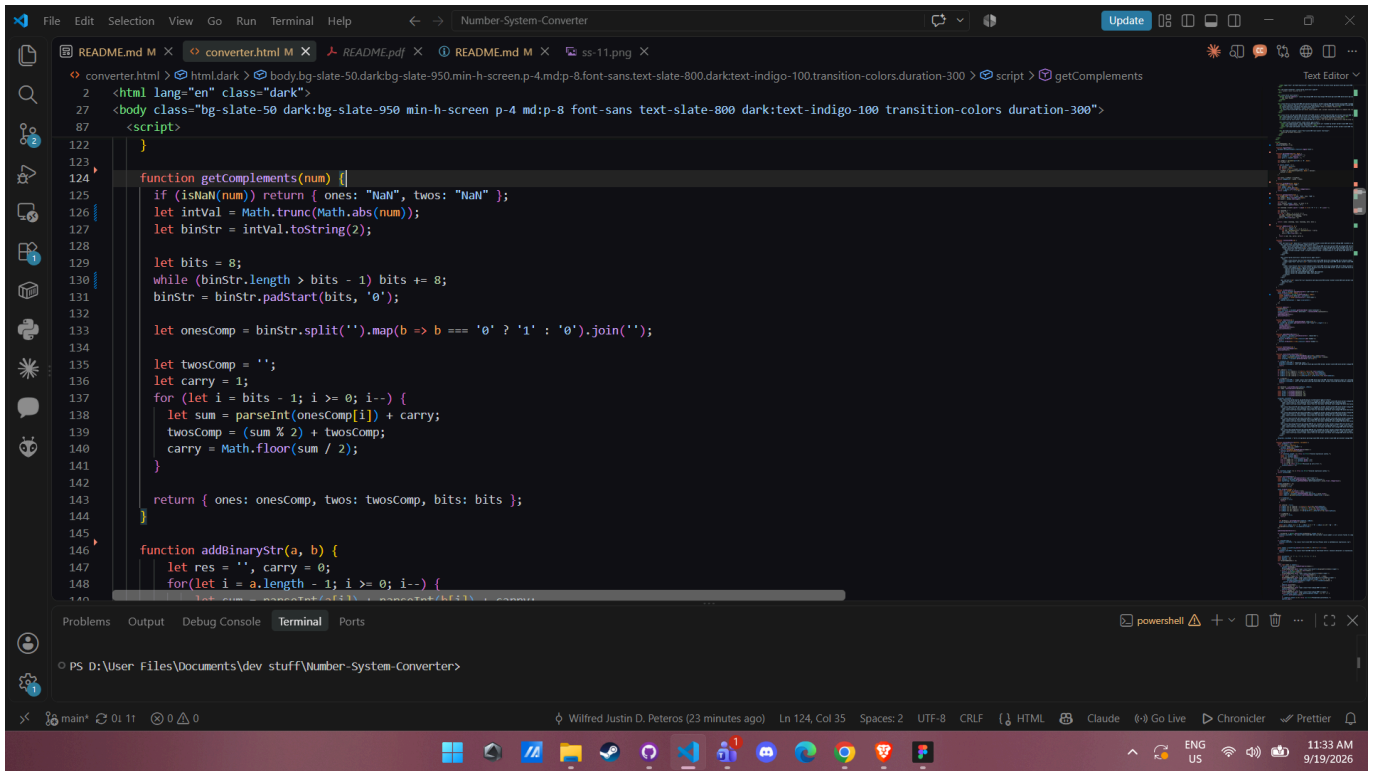
```
File Edit Selection View Go Run Terminal Help
Number-System-Converter > converter.html > html:dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
189 function convertIndividualNumber(id) {
216   const octStr = formatBase(decValue, 8);
217   const decStr = formatBase(decValue, 10);
218   const hexStr = formatBase(decValue, 16);
219
220   outputDiv.innerHTML = `
221     <div class="grid grid-cols-2 lg:grid-cols-4 gap-3 w-full">
222       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
223         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Binary</span>
224         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${binStr}</span></div>
225       </div>
226       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
227         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Octal</span>
228         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${octStr}</span></div>
229       </div>
230       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
231         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Decimal</span>
232         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${decStr}</span></div>
233       </div>
234       <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
235         <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Hexadecimal</span>
236         <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${hexStr}</span></div>
237       </div>
238     </div>
239   `;
240   outputDiv.className = "mt-4 p-3 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] transition-colors";
241 }
242
243 function evaluatePostfix(postfix, variables) {
244   const valStack = [];
245   for (let token of postfix) {
246     if (typeof token === 'number') {
247       valStack.push(token);
248     } else if (variables.hasOwnProperty(token)) {
249       valStack.push(variables[token]);
250     } else {
251       if (valStack.length < 2) throw new Error("Invalid expression syntax.");
252       const b = valStack.pop();
253       const a = valStack.pop();
254       if (token === '+') valStack.push(a + b);
255       else if (token === '-') valStack.push(a - b);
256       else if (token === '*') valStack.push(a * b);
257       else if (token === '/') {
258         if (b === 0) throw new Error("Division by zero error.");
259       }
260     }
261   }
262   return valStack[0];
263 }
```

```
File Edit Selection View Go Run Terminal Help
Number-System-Converter > converter.html > html:dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
243 function evaluatePostfix(postfix, variables) {
260 }
261 }
262 }
263 if (valStack.length !== 1) throw new Error("Invalid expression syntax.");
264 return valStack[0];
265 }
266
267 function calculateTotal() {
268 const cases = document.querySelectorAll('[id~="case-"]');
269 const totalDiv = document.getElementById('final-total-container');
270 const exprString = document.getElementById('math-expression').value.trim().toLowerCase();
271
272 let variables = {};
273 let displayStrs = {};
274 let allValid = true;
275
276 cases.forEach(caseEl => {
277 const id = caseEl.id.split('-')[1];
278 const varName = caseEl.getAttribute('data-var');
279 const rawValue = document.getElementById(`val-${id}`).value.trim();
280 const inBase = parseInt(document.getElementById(`inBase-${id}`).value);
281
282 if (!rawValue) {
283 allValid = false;
284 return;
285 }
286
287 let isValid = false;
288 if (inBase === 2) isValid = /^[01]+$/i.test(rawValue);
289 if (inBase === 8) isValid = /^[0-7]+$/i.test(rawValue);
290 if (inBase === 10) isValid = /^[0-9]+$/i.test(rawValue);
291 if (inBase === 16) isValid = /^[0-9a-fA-F]+$/i.test(rawValue);
292
293 if (!isValid) {
294 allValid = false;
295 return;
296 }
297
298 let decValue = parseDecimal(rawValue, inBase);
299 variables[varName] = decValue;
300
301 const sub = inBase === 2 ? '2' : inBase === 8 ? '8' : inBase === 10 ? '10' : '16';
302 displayStrs[varName] = `(${rawValue})${sub}`;
303 });
304
305 if (!allValid || Object.keys(variables).length === 0) {
306 totalDiv.innerHTML = "<p class='text-slate-400 text-lg'>Enter valid numbers in all active fields to compute.</p>";
307 return;
308 }
309
310 if (!exprString) {
311 totalDiv.innerHTML = "<p class='text-slate-400 text-lg'>Please enter a mathematical expression.</p>";
312 return;
313 }
314
315 // Regex updated to parse direct decimal numbers typed in the expression field
316 const tokens = exprString.match(/[A-Z]|\d|\+|\-|\*|\/|\^|\%|\(|\)/g);
317 if (!tokens) {
318 totalDiv.innerHTML = "<p class='text-red-400 text-xl font-bold'>Error: Invalid characters in expression.</p>";
319 return;
320 }
321
322 const precedence = { '+': 1, '-': 1, '*': 2, '/': 2 };
323 const postfix = [];
324 const opStack = [];
325 let formattedEqTokens = [];
326
327 try {
328 for (let token of tokens) {
329 if (variables.hasOwnProperty(token)) {
330 postfix.push(token);
331 formattedEqTokens.push("<span class='text-white'>${displayStrs[token]}</span>");
332 } else if (/^[0-9]*$/i.test(token)) {
333 postfix.push(Number(token));
334 formattedEqTokens.push("<span class='text-white'>${token}</span>");
335 } else if (!['+', '-', '*', '/'].includes(token)) {
336 let symbol = token === '^' ? '^' : token === '/' ? '÷' : token;
337 formattedEqTokens.push("<span class='text-pink-400 mx-1'>${symbol}</span>");
338 while (opStack.length > 0 && opStack[opStack.length - 1] !== '(' &&
339 precedence[opStack[opStack.length - 1]] >= precedence[token]) {
340 postfix.push(opStack.pop());
341 }
342 opStack.push(token);
343 } else if (token === '(') {
344 formattedEqTokens.push("<span class='text-indigo-400'>(</span>");
345 opStack.push(token);
346 }
347 }
348 } catch {
349 }
350
351 while (opStack.length > 0) {
352 postfix.push(opStack.pop());
353 }
354
355 formattedEqTokens.push("<span class='text-white'>= </span>");
356 totalDiv.innerHTML = formattedEqTokens.join('');
357 }
```

```
File Edit Selection View Go Run Terminal Help
Number-System-Converter > converter.html > html:dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
267 function calculateTotal() {
270 cases.forEach(caseEl => {
303 });
304
305 if (!allValid || Object.keys(variables).length === 0) {
306 totalDiv.innerHTML = "<p class='text-slate-400 text-lg'>Enter valid numbers in all active fields to compute.</p>";
307 return;
308 }
309
310 if (!exprString) {
311 totalDiv.innerHTML = "<p class='text-slate-400 text-lg'>Please enter a mathematical expression.</p>";
312 return;
313 }
314
315 // Regex updated to parse direct decimal numbers typed in the expression field
316 const tokens = exprString.match(/[A-Z]|\d|\+|\-|\*|\/|\^|\%|\(|\)/g);
317 if (!tokens) {
318 totalDiv.innerHTML = "<p class='text-red-400 text-xl font-bold'>Error: Invalid characters in expression.</p>";
319 return;
320 }
321
322 const precedence = { '+': 1, '-': 1, '*': 2, '/': 2 };
323 const postfix = [];
324 const opStack = [];
325 let formattedEqTokens = [];
326
327 try {
328 for (let token of tokens) {
329 if (variables.hasOwnProperty(token)) {
330 postfix.push(token);
331 formattedEqTokens.push("<span class='text-white'>${displayStrs[token]}</span>");
332 } else if (/^[0-9]*$/i.test(token)) {
333 postfix.push(Number(token));
334 formattedEqTokens.push("<span class='text-white'>${token}</span>");
335 } else if (!['+', '-', '*', '/'].includes(token)) {
336 let symbol = token === '^' ? '^' : token === '/' ? '÷' : token;
337 formattedEqTokens.push("<span class='text-pink-400 mx-1'>${symbol}</span>");
338 while (opStack.length > 0 && opStack[opStack.length - 1] !== '(' &&
339 precedence[opStack[opStack.length - 1]] >= precedence[token]) {
340 postfix.push(opStack.pop());
341 }
342 opStack.push(token);
343 } else if (token === '(') {
344 formattedEqTokens.push("<span class='text-indigo-400'>(</span>");
345 opStack.push(token);
346 }
347 }
348 } catch {
349 }
350
351 while (opStack.length > 0) {
352 postfix.push(opStack.pop());
353 }
354
355 formattedEqTokens.push("<span class='text-white'>= </span>");
356 totalDiv.innerHTML = formattedEqTokens.join('');
357 }
```

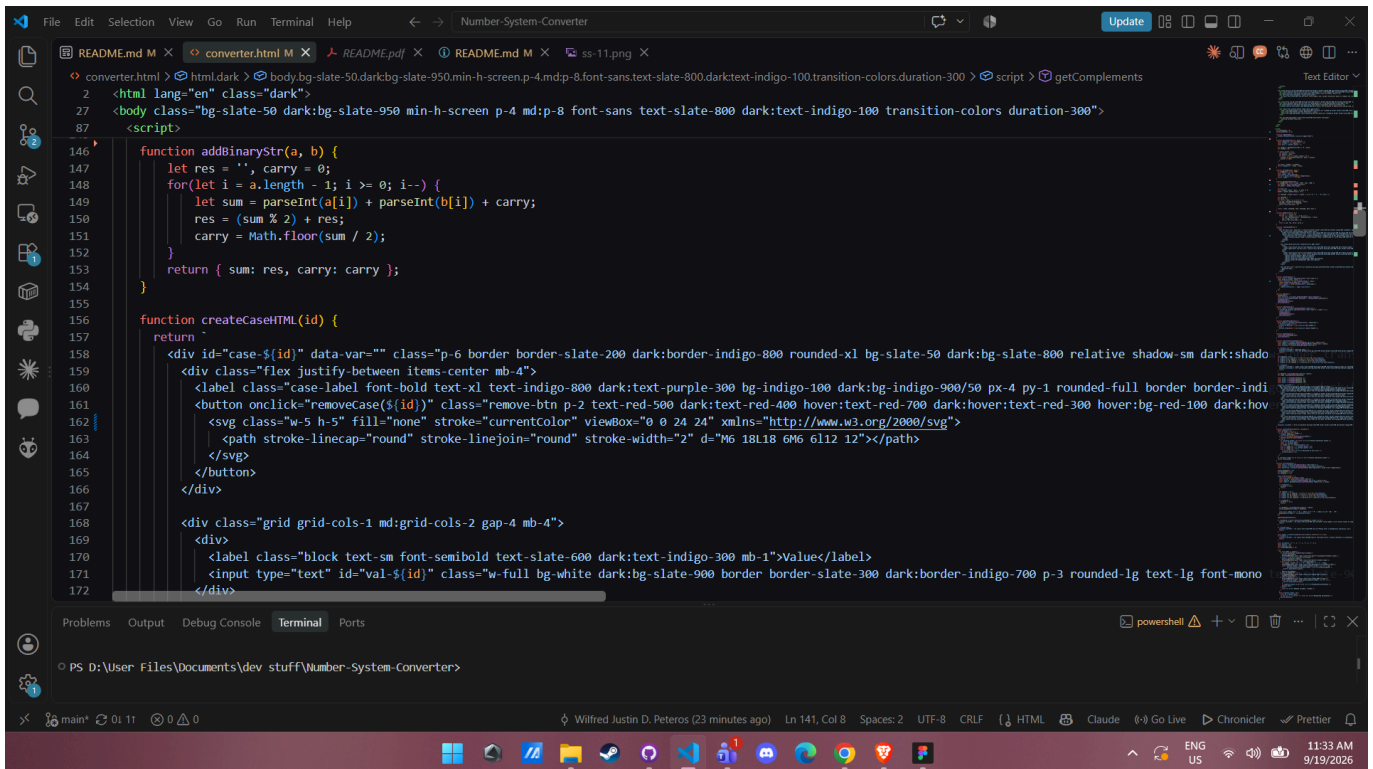
```
Number-System-Converter > converter.html > html.dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
267 function calculateTotal() {
342   opStack.push(token);
343   } else if (token === '(') {
344     formattedTokens.push('<span class="text-indigo-400"></span>');
345     opStack.push(token);
346   } else if (token === ')') {
347     formattedTokens.push('<span class="text-indigo-400"></span>');
348     while (opStack.length > 0 && opStack[opStack.length - 1] !== '(') {
349       postfix.push(opStack.pop());
350     }
351     if (opStack.length === 0) throw new Error("Mismatched parentheses.");
352     opStack.pop();
353   } else {
354     throw new Error("Unknown variable: ${token}");
355   }
356 }
357 while (opStack.length > 0) {
358   const op = opStack.pop();
359   if (op === '(' || op === ')') throw new Error("Mismatched parentheses.");
360   postfix.push(op);
361 }
362
363 const finalResult = evaluatePostfix(postfix, variables);
364
365 const binStr = formatBase(finalResult, 2);
366 const octStr = formatBase(finalResult, 8);
367 const decStr = formatBase(finalResult, 10);
368 const hexStr = formatBase(finalResult, 16);
369
370 totalDiv.innerHTML = `
371 <div class="mb-6 p-4 bg-slate-800 rounded-lg border border-slate-700 shadow-inner overflow-x-auto">
372   <p class="text-indigo-300 text-sm font-bold uppercase tracking-wider mb-2">Equation Breakdown</p>
373   <p class="text-xl md:text-2xl font-mono whitespace-nowrap">${formattedEqTokens.join('')} <span class="text-pink-400 mx-2">=</span></p>
374 </div>
375
376 <div class="grid grid-cols-1 md:grid-cols-2 gap-4">
377   <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
378     <span class="block text-sm font-bold text-slate-400 mb-1">Binary</span>
379     <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-green-400 break-all">${binStr}</span></div>
380   </div>
381   <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
382     <span class="block text-sm font-bold text-slate-400 mb-1">Octal</span>
383     <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-blue-400 break-all">${octStr}</span></div>
384   </div>
385   <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
386     <span class="block text-sm font-bold text-slate-400 mb-1">Decimal</span>
387     <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-yellow-400 break-all">${decStr}</span></div>
388   </div>
389   <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
390     <span class="block text-sm font-bold text-slate-400 mb-1">Hexadecimal</span>
391     <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-pink-400 break-all">${hexStr}</span></div>
392   </div>
393 </div>
394
395 }
396 } catch (err) {
397   totalDiv.innerHTML = `<p class="text-red-400 text-xl font-bold">Error: ${err.message}</p>`;
398 }
399
400 addCase();
401 addCase();
402 addCase();
403 </script>
404 </body>
405 </html>
```

```
Number-System-Converter > converter.html > html.dark > head > script > theme > extend
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
71 <script>
267 function calculateTotal() {
379   <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-green-400 break-all">${binStr}</span></div>
380 </div>
381 <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
382   <span class="block text-sm font-bold text-slate-400 mb-1">Octal</span>
383   <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-blue-400 break-all">${octStr}</span></div>
384 </div>
385 <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
386   <span class="block text-sm font-bold text-slate-400 mb-1">Decimal</span>
387   <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-yellow-400 break-all">${decStr}</span></div>
388 </div>
389 <div class="bg-slate-800 p-4 rounded-xl border border-slate-700 text-left overflow-hidden">
390   <span class="block text-sm font-bold text-slate-400 mb-1">Hexadecimal</span>
391   <div class="overflow-x-auto"><span class="text-xl md:text-2xl font-mono font-bold text-pink-400 break-all">${hexStr}</span></div>
392 </div>
393 </div>
394
395 }
396 } catch (err) {
397   totalDiv.innerHTML = `<p class="text-red-400 text-xl font-bold">Error: ${err.message}</p>`;
398 }
399
400 addCase();
401 addCase();
402 addCase();
403 </script>
404 </body>
405 </html>
```



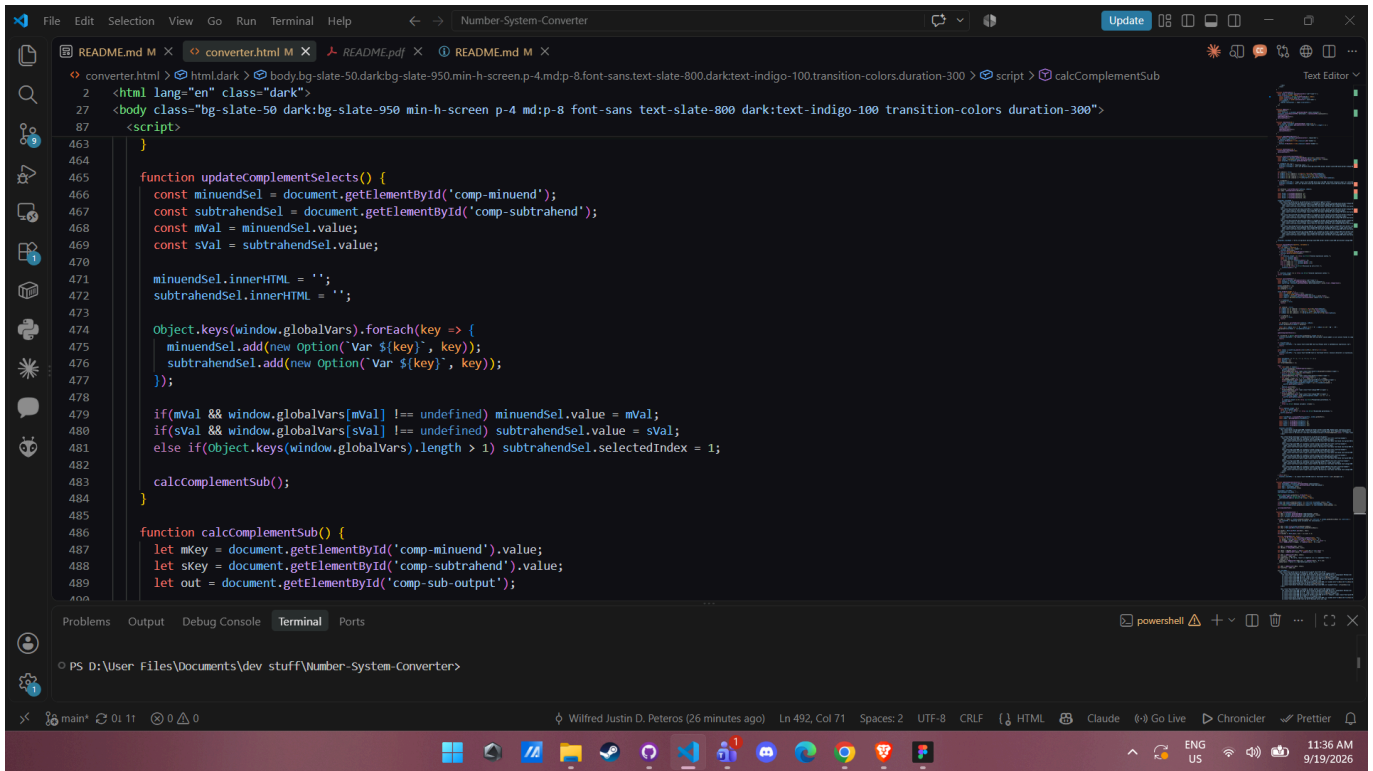
This screenshot shows the VS Code editor with the file `converter.html` open. The code defines a `getComplements` function that takes a number and returns an object with `ones`, `twos`, and `bits` properties. The function uses binary conversion and bitwise operations to calculate the complements. The editor interface includes a sidebar with file explorer, a top bar with search and view icons, and a bottom status bar showing the current file path and language mode.

```
122
123
124 function getComplements(num) {
125   if (isNaN(num)) return { ones: "NaN", twos: "NaN" };
126   let intVal = Math.trunc(Math.abs(num));
127   let binStr = intVal.toString(2);
128
129   let bits = 8;
130   while (binStr.length > bits - 1) bits += 8;
131   binStr = binStr.padStart(bits, '0');
132
133   let onesComp = binStr.split('').map(b => b === '0' ? '1' : '0').join('');
134
135   let twosComp = '';
136   let carry = 1;
137   for (let i = bits - 1; i >= 0; i--) {
138     let sum = parseInt(onesComp[i]) + carry;
139     twosComp = (sum % 2) + twosComp;
140     carry = Math.floor(sum / 2);
141   }
142
143   return { ones: onesComp, twos: twosComp, bits: bits };
144 }
145
146 function addBinaryStr(a, b) {
147   let res = '', carry = 0;
148   for (let i = a.length - 1; i >= 0; i--) {
```



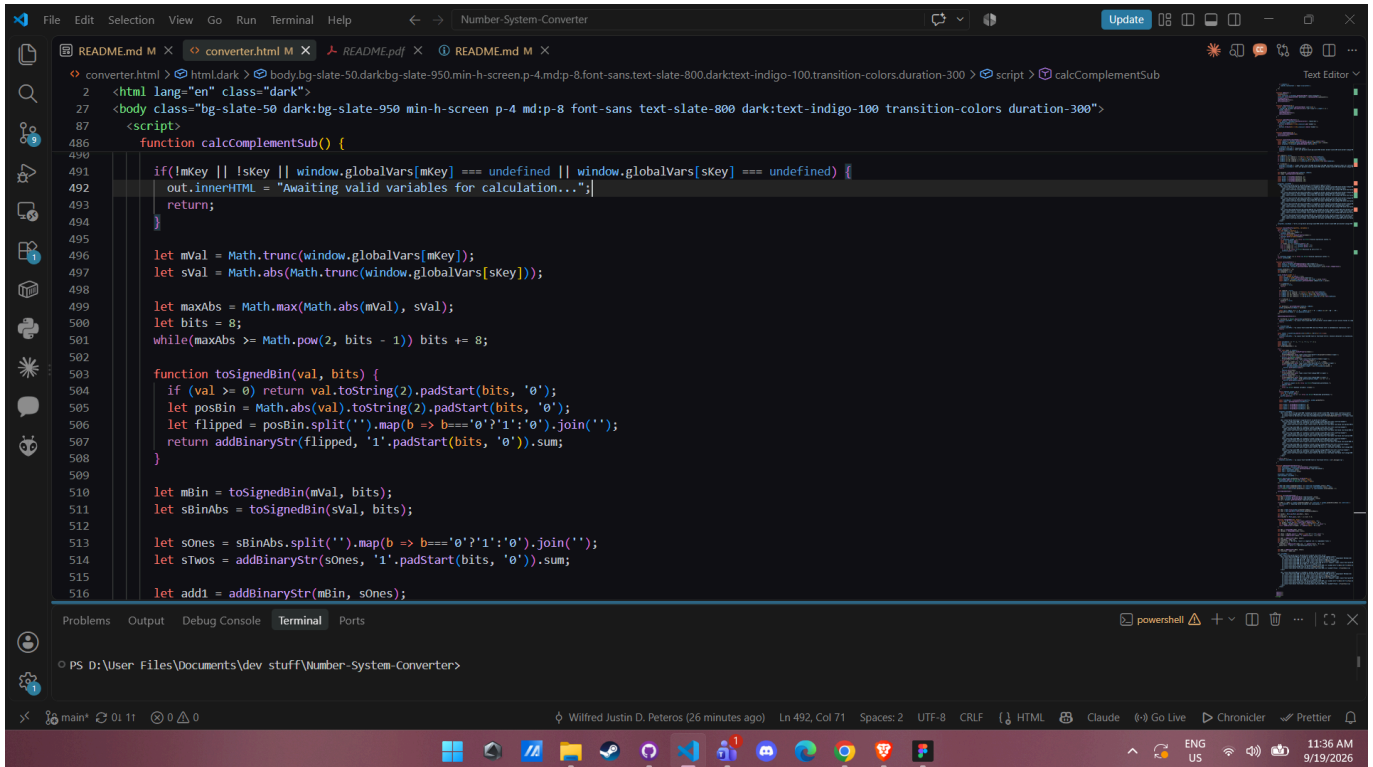
This screenshot shows the VS Code editor with the file `converter.html` open. The code defines a `createCaseHTML` function that generates HTML for a case input. It includes a button to remove the case and a text input field. The function uses template literals to construct the HTML string. The editor interface is consistent with the previous screenshot, showing the same file explorer and status bar.

```
146 function addBinaryStr(a, b) {
147   let res = '', carry = 0;
148   for (let i = a.length - 1; i >= 0; i--) {
149     let sum = parseInt(a[i]) + parseInt(b[i]) + carry;
150     res = (sum % 2) + res;
151     carry = Math.floor(sum / 2);
152   }
153   return { sum: res, carry: carry };
154 }
155
156 function createCaseHTML(id) {
157   return `
158     <div id="case-${id}" data-var="" class="p-6 border border-slate-200 dark:border-indigo-800 rounded-xl bg-slate-50 dark:bg-slate-800 relative shadow-sm dark:shado
159     <div class="flex justify-between items-center mb-4">
160       <label class="case-label font-bold text-xl text-indigo-800 dark:text-purple-300 bg-indigo-100 dark:bg-indigo-900/50 px-4 py-1 rounded-full border border-indi
161       <button onclick="removeCase('${id}')" class="remove-btn p-2 text-red-500 dark:text-red-400 hover:text-red-700 dark:hover:text-red-300 hover:bg-red-100 dark:hov
162       <svg class="w-5 h-5 fill="none" stroke="currentcolor" viewBox="0 0 24 24" xmlns="http://www.w3.org/2000/svg">
163         <path stroke-linecap="round" stroke-linejoin="round" stroke-width="2" d="M6 18L18 6M6 12L12 6"/>
164       </svg>
165     </button>
166   </div>
167
168   <div class="grid grid-cols-1 md:grid-cols-2 gap-4 mb-4">
169     <div>
170       <label class="block text-sm font-semibold text-slate-600 dark:text-indigo-300 mb-1">Value</label>
171       <input type="text" id="val-${id}" class="w-full bg-white dark:bg-slate-900 border border-slate-300 dark:border-indigo-700 p-3 rounded-lg text-lg font-mono
172     </div>
```



This screenshot shows the Visual Studio Code editor with the file `converter.html` open. The editor is displaying the `updateComplementSelects` function, which is responsible for updating the HTML content of two select elements, `minuendSel` and `subtrahendSel`, based on the values of `mVal` and `sVal`. The function also updates the `selectedIndex` of `subtrahendSel` if `subtrahendSel` is not empty. The `calcComplementSub` function is also visible, which calculates the result of the subtraction and updates the `out` element.

```
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
87 <script>
463 }
464
465 function updateComplementSelects() {
466   const minuendSel = document.getElementById('comp-minuend');
467   const subtrahendSel = document.getElementById('comp-subtrahend');
468   const mVal = minuendSel.value;
469   const sVal = subtrahendSel.value;
470
471   minuendSel.innerHTML = '';
472   subtrahendSel.innerHTML = '';
473
474   Object.keys(window.globalVars).forEach(key => {
475     minuendSel.add(new Option('Var ${key}', key));
476     subtrahendSel.add(new Option('Var ${key}', key));
477   });
478
479   if(mVal && window.globalVars[mVal] !== undefined) minuendSel.value = mVal;
480   if(sVal && window.globalVars[sVal] !== undefined) subtrahendSel.value = sVal;
481   else if(Object.keys(window.globalVars).length > 1) subtrahendSel.selectedIndex = 1;
482
483   calcComplementSub();
484 }
485
486 function calcComplementSub() {
487   let mKey = document.getElementById('comp-minuend').value;
488   let sKey = document.getElementById('comp-subtrahend').value;
489   let out = document.getElementById('comp-sub-output');
```



This screenshot shows the Visual Studio Code editor with the file `converter.html` open. The editor is displaying the `calcComplementSub` function, which calculates the result of the subtraction and updates the `out` element. The function uses `Math.trunc` to truncate the values of `mVal` and `sVal` to the nearest integer. It then calculates the maximum absolute value of `mVal` and `sVal` and determines the number of bits required to represent the result. The function then converts the values to binary and calculates the result using bitwise operations.

```
486 function calcComplementSub() {
490   if(!mKey || !sKey || window.globalVars[mKey] === undefined || window.globalVars[sKey] === undefined) {
491     out.innerHTML = "Awaiting valid variables for calculation...";
492     return;
493   }
494
495   let mVal = Math.trunc(window.globalVars[mKey]);
496   let sVal = Math.trunc(window.globalVars[sKey]);
497
498   let maxAbs = Math.max(Math.abs(mVal), Math.abs(sVal));
499   let bits = 8;
500   while(maxAbs >= Math.pow(2, bits - 1)) bits += 8;
501
502   function toSignedBin(val, bits) {
503     if (val >= 0) return val.toString(2).padStart(bits, '0');
504     let posBin = Math.abs(val).toString(2).padStart(bits, '0');
505     let flipped = posBin.split('').map(b => b === '0' ? '1' : '0').join('');
506     return addBinaryStr(flipped, '1'.padStart(bits, '0')).sum;
507   }
508
509   let mBin = toSignedBin(mVal, bits);
510   let sBinAbs = toSignedBin(sVal, bits);
511
512   let sOnes = sBinAbs.split('').map(b => b === '0' ? '1' : '0').join('');
513   let sTwos = addBinaryStr(sOnes, '1'.padStart(bits, '0')).sum;
514
515   let add1 = addBinaryStr(mBin, sTwos);
516 }
```

[illegible]

The screenshot displays a Windows 11 desktop environment. The primary focus is the Visual Studio Code (VS Code) editor, which is open to a file named 'converter.html'. The editor's interface includes a sidebar on the left with icons for Explorer, Search, and Source Control. The main editor area shows a mix of HTML and JavaScript code. The HTML part defines a web page with a dark theme, using classes like 'dark', 'text-white', and 'text-emerald'. The JavaScript part includes a function 'calcComplementsSub' that calculates the complement of a number. The code is syntax-highlighted, and line numbers are visible on the left. To the right of the editor is a 'Text Editor' pane showing a preview of the web page. The preview shows a dark-themed interface with text and buttons, reflecting the styles defined in the HTML. Below the editor is a 'Terminal' pane, which is currently empty. The Windows taskbar is visible at the bottom, showing icons for various applications like File Explorer, VS Code, and a web browser. The system clock in the bottom right corner indicates the time is 11:36 AM on 9/19/2026.

```
File Edit Selection View Go Run Terminal Help Number-System-Converter
ion-colors:duration-300 > div.mt-8.p-8.bg-slate-800.dark:bg-slate-950.border-2.border-emerald-500.dark:border-emerald-600.rounded-2xl.shadow-xl.relative.overflow-hidden > div#comp-sub-output.text-slate-300.text-center.font-mono
2 m1 lang="en" class="dark">
27 dy class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
29 div class="max-w-6xl mx-auto">
41 <div class="bg-white dark:bg-slate-900 p-6 md:p-8 rounded-2xl shadow-xl border border-slate-200 dark:border-indigo-900 transition-colors duration-300">
60 <div class="mt-12 p-8 bg-slate-900 dark:bg-black border-2 border-indigo-500 dark:border-purple-600 rounded-2xl shadow-2xl relative overflow-hidden">
65 </div>
66 </div>
67
68 <div class="mt-8 p-8 bg-slate-800 dark:bg-slate-950 border-2 border-emerald-500 dark:border-emerald-600 rounded-2xl shadow-xl relative overflow-hidden">
69 <div class="absolute top-0 left-0 w-full h-1 bg-gradient-to-r from-emerald-400 to-teal-500"></div>
70 <h2 class="text-2xl font-bold text-white mb-2 text-center">Complement Subtraction Method</h2>
71 <p class="text-sm text-center text-emerald-400 mb-6">Select two variables to demonstrate step-by-step subtraction (Minuend - Subtrahend) using complements. Uses abs
72
73 <div class="flex justify-center items-center gap-4 mb-6">
74 <select id="comp-minuend" class="bg-slate-700 text-white p-3
75 rounded-lg border border-slate-600 focus:ring-2 focus:ring-emerald-500 font-bold text-lg" onchange="calcComplementSub()"></select>
76 <span class="text-white text-3xl font-bold"></span>
77 <select id="comp-subtrahend" class="bg-slate-700 text-white p-3 rounded-lg
78 border border-slate-600 focus:ring-2 focus:ring-emerald-500 font-bold text-lg" onchange="calcComplementSub()"></select>
79 </div>
80
81 <div id="comp-sub-output" class="text-slate-300 text-center font-mono">
82 |Awaiting variable selection...
83 </div>
84 </div>
85
86 </div>
87 /div>
88
89 script>
```

```
File Edit Selection View Go Run Terminal Help Number-System-Converter
converter.html > html.dark > body.bg-slate-50.dark:bg-slate-950.min-h-screen.p-4.md:p-8.font-sans.text-slate-800.dark:text-indigo-100.transition-colors.duration-300 > script > evaluatePostfix
2 <html lang="en" class="dark">
27 <body class="bg-slate-50 dark:bg-slate-950 min-h-screen p-4 md:p-8 font-sans text-slate-800 dark:text-indigo-100 transition-colors duration-300">
89 <script>
238 function convertIndividualNumber(id) {
239 <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
280 <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Decimal</span>
281 <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${decStr}</span></div>
282 </div>
283 <div class="bg-slate-50 dark:bg-slate-900 p-3 rounded-lg border border-slate-200 dark:border-indigo-800 min-w-0 flex flex-col">
284 <span class="text-xs font-semibold text-slate-500 dark:text-indigo-400 mb-1">Hexadecimal</span>
285 <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-700 dark:text-purple-300 break-all">${hexStr}</span></div>
286 </div>
287 <div class="bg-indigo-50 dark:bg-indigo-950/40 p-3 rounded-lg border border-indigo-200 dark:border-indigo-800 min-w-0 flex flex-col">
288 <span class="text-xs font-semibold text-indigo-600 dark:text-indigo-400 mb-1">1's Comp (Int)</span>
289 <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-800 dark:text-purple-200 break-all">${comps.ones}</span></div>
290 </div>
291 <div class="bg-indigo-50 dark:bg-indigo-950/40 p-3 rounded-lg border border-indigo-200 dark:border-indigo-800 min-w-0 flex flex-col">
292 <span class="text-xs font-semibold text-indigo-600 dark:text-indigo-400 mb-1">2's Comp (Int)</span>
293 <div class="overflow-x-auto"><span class="text-sm font-mono font-bold text-indigo-800 dark:text-purple-200 break-all">${comps.twos}</span></div>
294 </div>
295 </div>
296
297 outputDiv.className = "mt-4 p-3 bg-white dark:bg-slate-950 border border-slate-200 dark:border-indigo-900 rounded-lg min-h-[80px] transition-colors";
298 }
299
300 function evaluatePostfix(postfix, variables) {
301 const valStack = [];
302 for (let token of postfix) {
303 if (typeof token === 'number') {
304 valStack.push(token);
305 } else if (variables.hasOwnProperty(token)) {
```